perplexity

```
# \<q>String suche = suchtext.toLowerCase();\</q> was mache ich dann mit dieser Varia
```

Du verwendest `suche` **im** `contains(...)`**-Teil** anstelle von `suchtext.toLowerCase()`:

```
public void ticketsSuchen(String suchtext) {
    String suche = suchtext.toLowerCase();

    tickets.stream()
            .filter(ticket -> ticket.getTitel()
                    .toLowerCase()
                    .contains(suche))
            .forEach(ticket -> System.out.println(ticket));
}
```

`suche` enthält also den einmal umgewandelten Suchbegriff. Dadurch muss Java `suchtext.toLowerCase()` nicht bei jedem einzelnen Ticket erneut ausführen.

Beispiel: Eingabe `SERVER` wird in `suche` zu `server`; anschließend wird geprüft, ob jeder kleingeschriebene Titel `server` enthält.[1]

Welche Stelle in deiner bisherigen `filter`-Zeile ersetzt du also durch `suche`?

⁂

Z4: Erstes kritisches Ticket

Z4 soll für einen bestimmten Bearbeiter das **erste Ticket** finden, das gleichzeitig zu diesem Bearbeiter gehört und kritisch ist, also Priorität 1 hat. Weil möglicherweise kein passendes Ticket existiert, liefert `findFirst()` ein `Optional<Ticket>`.[15]

Die Methode gehört gut in `TicketSystem` und braucht den Bearbeiternamen als Parameter:

```
public void erstesKritischesTicket(String bearbeiter) {
    // Stream-Suche kommt hier hinein
}
```

Der erste Schritt ist wieder der Stream über alle Tickets:

```
tickets.stream()
```

Welches Kriterium muss der erste `filter(...)` prüfen, damit nur Tickets dieses Bearbeiters übrig bleiben?

⁂

.filter(ticket -> ticket.getBearbeiter())

Fast. Ein `filter(...)` braucht eine Bedingung, die `true` oder `false` ergibt. `ticket.getBearbeiter()` allein liefert aber einen `String`.

Du musst den Bearbeiter des Tickets mit dem Methodenparameter `bearbeiter` vergleichen:

```
.filter(ticket -> ticket.getBearbeiter().equals(bearbeiter))
```

`equals(...)` prüft bei Strings den Inhalt, also zum Beispiel: „Ist der Bearbeiter dieses Tickets `Anna Berger`?“ `getBearbeiter()` liefert dazu den Bearbeiternamen. [16]

Wie lautet der zweite Filter, der nur kritische Tickets mit Priorität 1 übrig lässt?

⁂

findFirst

`findFirst()` kommt **nach beiden Filtern.** Es sucht dann das erste Ticket, das alle Bedingungen erfüllt.

Der zweite Filter prüft zuerst, ob die Priorität 1 ist:

```
.filter(ticket -> ticket.getPrioritaet() == 1)
```

Danach kommt:

```
.findFirst()
```

Der Aufbau beginnt damit:

```
tickets.stream()
        .filter(ticket -> ticket.getBearbeiter().equals(bearbeiter))
        .filter(ticket -> ticket.getPrioritaet() == 1)
        .findFirst();
```

`findFirst()` liefert ein `Optional<Ticket>`, weil es eventuell kein kritisches Ticket für diesen Bearbeiter gibt. [18] [19]

Wie würdest du die Variable deklarieren, in der du das Ergebnis von `findFirst()` speicherst?

⁂

erstesKritischesTicket

Der Name passt gut. Es fehlt noch der **Datentyp**, denn `findFirst()` liefert kein direktes `Ticket`, sondern ein `Optional<Ticket>`.

```
Optional<Ticket> erstesKritischesTicket = tickets.stream()
        .filter(ticket -> ticket.getBearbeiter().equals(bearbeiter))
        .filter(ticket -> ticket.getPrioritaet() == 1)
        .findFirst();
```

Dafür brauchst du oben in `TicketSystem.java` außerdem:

```
import java.util.Optional;
```

Ein `Optional<Ticket>` bedeutet: Es kann ein Ticket enthalten — oder leer sein, wenn kein kritisches Ticket für diesen Bearbeiter existiert. [20]

Welche Methode kannst du jetzt auf `erstesKritischesTicket` aufrufen, um zu prüfen, ob tatsächlich ein Ticket gefunden wurde?

⁂

public void erstesKritischesTicket(String bearbeiter) {

Optional<Ticket> erstesKritischesTicket = tickets.stream()
.filter(ticket -> ticket.getBearbeiter().equals(bearbeiter))
.filter(ticket -> ticket.getPrioritaet() == 1)
.findFirst();
}

Das ist bis hierhin korrekt. Dein `Optional<Ticket>` enthält entweder das erste kritische Ticket dieses Bearbeiters oder ist leer. [21]

Jetzt fehlt die Ausgabe für beide Fälle. Prüfe zuerst, ob ein Wert vorhanden ist:

```
if (erstesKritischesTicket.isPresent()) {
    // gefundenes Ticket ausgeben
} else {
    // Meldung ausgeben
}
```

`isPresent()` liefert `true`, wenn das `Optional` tatsächlich ein Ticket enthält.

Welche Methode rufst du innerhalb des `if` auf, um das enthaltene Ticket aus dem `Optional` zu holen?

⁂

Kannst du mir helfen?

Ja. Denk an `Optional` wie an eine Schachtel: Erst mit `isPresent()` prüfen, ob etwas darin liegt; anschließend kannst du es mit `get()` herausnehmen. Z4 verlangt genau diesen Fall: Ticket ausgeben oder eine Meldung, falls keines gefunden wurde.[22]

```
if (erstesKritischesTicket.isPresent()) {
    System.out.println(erstesKritischesTicket.get());
} else {
    System.out.println("Keine kritischen Tickets");
}
```

Setze diesen Block direkt **nach** deiner `findFirst()`-Zeile in die Methode ein. `get()` ist hier sicher, weil es nur im `if` ausgeführt wird, wenn `isPresent()` wahr ist.

Wie würdest du die Meldung im `else` noch verbessern, damit klar wird, für welchen Bearbeiter kein kritisches Ticket gefunden wurde?

⁂

System.out.println("Für " + bearbeiter + " wurden keine kritischen Tickets gefunden.");

Ja, das ist eine gute und verständliche Ausgabe:

```
System.out.println("Für " + bearbeiter
        + " wurden keine kritischen Tickets gefunden.");
```

Damit erhält der Benutzer nicht nur die Information, dass kein Treffer existiert, sondern auch, auf welchen Bearbeiter sich die Suche bezog. Das erfüllt die geforderte Ausgabe für den Fall eines leeren `Optional`.[23]

Wie lautet deine vollständige Methode jetzt, wenn du Stream-Suche, `if` und `else` zusammensetzt?

⁂

public void erstesKritischesTicket(String bearbeiter) {

Optional<Ticket> erstesKritischesTicket = tickets.stream()
.filter(ticket -> ticket.getBearbeiter().equals(bearbeiter))
.filter(ticket -> ticket.getPrioritaet() == 1)
.findFirst();
if (erstesKritischesTicket.isPresent()) {
System.out.println(erstesKritischesTicket.get());
} else {

```
System.out.println("Für " + bearbeiter + " wurden keine kritischen Tickets gefunden.");
}
}
```

Das ist vollständig korrekt. Die Methode:

- filtert nach Bearbeiter,
- filtert nach Priorität 1,
- nimmt mit `findFirst()` den ersten Treffer,
- gibt das Ticket aus oder eine passende Meldung, falls kein Treffer vorhanden ist. [24]

Achte nur auf den Import, falls er noch fehlt:

```
import java.util.Optional;
```

Als nächstes musst du Z4 im Menü zugänglich machen, zum Beispiel mit einem neuen Fall `case "9":`, der nach dem Bearbeiternamen fragt und anschließend `system.erstesKritischesTicket(...)` aufruft.

Welche zwei Zeilen brauchst du in diesem `case`, um zuerst den Bearbeiter einzulesen und dann deine Methode aufzurufen?

⁂

Wo gehört Z4 hin?

Die **Methode selbst** gehört in `TicketSystem.java`, weil dort die Liste `tickets` verwaltet wird. Dort hast du sie bereits richtig geschrieben.

Damit sie tatsächlich ausgeführt wird, brauchst du zusätzlich einen Menüpunkt in `Main.java`. Das vorhandene Menü hat bisher nur die Fälle 1 bis 7 und 0; ergänze deshalb beispielsweise `case "9"` im `switch`. [25]

```
case "9":
    System.out.print("Bearbeiter: ");
    String bearbeiter = scanner.nextLine().trim();
    system.erstesKritischesTicket(bearbeiter);
    break;
```

Und in `menueAusgeben()`:

```
System.out.println("9 Erstes kritisches Ticket eines Bearbeiters");
```

Wichtig: Das setzt voraus, dass du am Anfang von `main` bereits ein Objekt wie `TicketSystem system = new TicketSystem();` erzeugt und mit den Beispieldaten befüllt hast. In deinem Ausgangsgerüst ist das noch als TODO markiert. [25]

In welche Klasse gehört nach dieser Aufteilung die **Logik zum Finden** und in welche Klasse gehört die **Eingabe über den Scanner**?

⁂

Variable 'bearbeiter' is already defined in the scope

Der Fehler bedeutet: In deinem `switch` gibt es bereits eine Variable mit dem Namen `bearbeiter`, vermutlich in `case "2"`. Bei einem `switch` teilen sich die `case`-Blöcke ohne zusätzliche geschweifte Klammern denselben Gültigkeitsbereich.

Verwende für Z4 einfach einen anderen, eindeutigen Namen:

```
case "9":
    System.out.print("Bearbeiter: ");
    String gesuchterBearbeiter = scanner.nextLine().trim();
    system.erstesKritischesTicket(gesuchterBearbeiter);
    break;
```

Alternativ könntest du jeden `case` in `{ ... }` einschließen. Der unterschiedliche Variablenname ist hier aber am einfachsten. Dein Grundgerüst enthält bereits den Bereich für „Neues Ticket anlegen“, wo vermutlich `bearbeiter` verwendet wird. [26]

Warum löst `gesuchterBearbeiter` den Fehler, obwohl sein Inhalt genauso ein Bearbeitername ist?

⁂

Teil 2 starten

Ja, gern — wieder Schritt für Schritt: Du schreibst jeweils den Code, wir prüfen und bauen darauf auf. Teil 2 verlangt mindestens je eine Aufgabe aus **Dokumentation**, **Erweiterung** und **Testen**; die empfohlene Reihenfolge ist D1, E1, T1, T2. [27]

Wir beginnen mit **D1: Javadoc**, weil dabei kein Programmverhalten geändert wird. Du dokumentierst `TicketSystem`, `Auswertung` und alle `public`-Methoden; Methoden mit Parametern brauchen `@param`, und Methoden mit Rückgabewert zusätzlich `@return`. [27]

Setze zunächst direkt über deine Klasse `TicketSystem` diesen kurzen Javadoc-Kommentar — ohne Umlaute im Quelltext:

```
/**
 * Verwaltet Tickets und deren Bearbeitung.
 */
public class TicketSystem {
```

Welche deiner `public`-Methoden in `TicketSystem` hat mindestens einen Parameter und wäre danach als erstes sinnvoll zu dokumentieren?

⁂

Wichtig: Was neu für euch ist, dürft ihr nachschlagen, im Internet suchen oder eine KI fragen. Lasst euch Dinge und Fehlermeldungen erklären, aber den Code schreibt ihr selbst. Wichtig ist, dass ihr jede Zeile versteht, die ihr schreibt.

Genau — daran halten wir uns strikt: Begriffe, Konzepte, Fehlermeldungen und einzelne nächste Schritte werden erklärt; **du schreibst den Code selbst** und setzt ihn in dein Projekt ein. Ich gebe dir also keine fertigen Komplettlösungen zum Kopieren, sondern wir erarbeiten jede Zeile gemeinsam und prüfen, ob du sie verstehst. [28]

Wenn du bei einer Zeile unsicher bist, sag einfach, was sie deiner Meinung nach macht. Dann klären wir genau diesen Punkt, bevor du weiterschreibst.

Sollen wir nach D1 weitermachen oder möchtest du direkt mit E1, also dem Kombinieren von `Predicate<Ticket>`-Filtern, beginnen?

⁂

D1: Javadoc

Gute Wahl. Für D1 dokumentierst du die Klassen `TicketSystem` und `Auswertung` sowie **jede `public`-Methode**. Jeder Kommentar beginnt mit `/**` und endet mit `*/`; bei Parametern kommt je ein `@param` dazu, bei Rückgabewerten ein `@return`. Außerdem sollen im Java-Quelltext keine Umlaute verwendet werden. [29]

Beginne in `TicketSystem.java` direkt über der Klassendeklaration. Überlege zuerst: **Wofür ist diese Klasse insgesamt zuständig?** Daraus formulierst du einen kurzen Satz, zum Beispiel im Muster:

```
/**
 * Verwaltet ...
 */
public class TicketSystem {
```

Schreib den Satz bitte selbst passend zu deiner Klasse und poste ihn hier.

⁂

Verwaltet Tickets und deren Bearbeitung.

Sehr gut. Das ist klar, treffend und enthält keine Umlaute. Setze es so direkt über `TicketSystem`:

```
/**
 * Verwaltet Tickets und deren Bearbeitung.
 */
public class TicketSystem {
```

Als Nächstes dokumentieren wir eine `public`-Methode mit Parametern. Welche Methode in deinem `TicketSystem` nimmt zum Beispiel ein `Ticket` oder einen Titel als Parameter entgegen?

hinzufuegen

Genau. `hinzufuegen(Ticket ticket)` braucht einen kurzen Beschreibungssatz und einen `@param`-Eintrag für `ticket`. Sie gibt nichts zurück, also braucht sie **kein** `@return`. Das ist genau die Vorgabe für D1.[30]

Schreibe über die Methode einen Kommentar nach diesem Muster und formuliere die beiden Texte selbst:

```
/**
 * ...
 *
 * @param ticket ...
 */
public void hinzufuegen(Ticket ticket) {
```

Was passiert in deiner Methode `hinzufuegen` zusätzlich, wenn das übergebene Ticket den Status `OFFEN` hat?

⁂

Fügt das Ticket zu offene Tickets hinzu

Fast. Ein Ticket wird in jedem Fall zur Liste **aller** Tickets hinzugefügt. **Nur wenn sein Status** `OFFEN` **ist**, kommt es zusätzlich in die Warteschlange `offeneTickets`.[31]

Für den Javadoc-Satz könntest du beide Aufgaben nennen, zum Beispiel in deiner eigenen Formulierung:

```
/**
 * Fuegt ein Ticket hinzu und reiht offene Tickets ein.
 *
 * @param ticket Das hinzuzufuegende Ticket.
 */
```

`hinzufuegen` hat keinen Rückgabewert (`void`), daher endet der Kommentar nach `@param`.

Welchen Unterschied würdest du in einem Satz zwischen der Liste aller Tickets und der Queue `offeneTickets` beschreiben?

⁂

Liste aller Tickets sind offene, gechlossene und in bearbeitung, also alle und Queue offeneTickets sind offene tickets der Reihen nach

Genau richtig. Präzise formuliert:

- Die Liste `tickets` enthält **alle** Tickets — offen, in Bearbeitung und erledigt.
- Die Queue `offeneTickets` enthält nur Tickets mit Status `OFFEN`, die noch bearbeitet werden müssen. Ursprünglich wäre ihre Reihenfolge FIFO; in deiner Erweiterung entscheidet stattdessen die Priorität. [32]

Diese Erklärung passt auch gut in deine spätere README bei D4.

Jetzt schreibe bitte den vollständigen Javadoc-Kommentar für `hinzufuegen(Ticket ticket)` selbst in deinen Code und poste ihn anschließend hier.

⁂

/**

- Fuegt ein Ticket hinzu und reiht offene Tickets ein.
-
- @param ticket Das hinzuzufuegende Ticket.
  */

Sehr gut. Der Kommentar ist vollständig: Beschreibung, ein `@param` für den einzigen Parameter und kein unnötiges `@return`, weil die Methode `void` ist. Er erfüllt damit die D1-Vorgaben. [33]

Als Nächstes nimm eine Methode, die **etwas zurückgibt**, zum Beispiel `anlegen(...)` oder `getTickets()`. Dort ergänzt du neben den `@param`-Zeilen zusätzlich eine `@return`-Zeile.

Welche dieser beiden Methoden möchtest du als Nächstes dokumentieren?

⁂

anlegen(...)

/**

- Legt ein neues Ticket mit der naechsten freien Id an.
-
- @param titel Titel des Tickets
- @param prioritaet Prioritaet 1 (kritisch) bis 4 (niedrig)
- @return Das neu angelgte Ticket
  */

Das Grundgerüst stimmt, aber hier fehlt etwas Wichtiges: Deine Methode `anlegen(...)` hat **vier Parameter**. Deshalb braucht der Javadoc-Kommentar vier `@param`-Zeilen — für `titel`, `kunde`, `bearbeiter` und `prioritaet`. D1 verlangt einen Eintrag pro Parameter. [34]

Diese Zeilen sind schon korrekt:

```
@param titel Titel des Tickets
@param prioritaet Prioritaet 1 (kritisch) bis 4 (niedrig)
```

Ergänze zwischen ihnen oder davor zwei passende Zeilen für `kunde` und `bearbeiter`. Außerdem ein kleiner Schreibfehler: `angelgte` sollte `angelegte` heißen.

Wie würdest du `@param kunde` in eigenen Worten beschreiben?

⁂

So stand es im Aufgabenblatt

Du hast recht — das Beispiel im Aufgabenblatt zeigt tatsächlich nur `titel` und `prioritaet`, obwohl die dargestellte Methode zusätzlich `kunde` und `bearbeiter` besitzt. [35]

Für eure Aufgabe gilt jedoch ausdrücklich: **„ein `@param` pro Parameter“**. Deshalb solltest du den Beispielkommentar sinnvoll vervollständigen, statt ihn exakt unvollständig zu übernehmen. [35]

Ergänze daher noch diese beiden Angaben in deinem Javadoc:

```
@param kunde Der Kunde des Tickets.
@param bearbeiter Der zustaendige Bearbeiter.
```

Warum ist es für jemanden, der deine Methode später benutzt, hilfreich, auch `kunde` und `bearbeiter` in der Dokumentation erklärt zu bekommen?

⁂

Weil diese auch im Methodenkopf stehen und wichtig sind

Richtig. Sie stehen nicht nur im Methodenkopf, sondern legen auch fest, **für welchen Kunden** und **bei welchem Bearbeiter** das neue Ticket angelegt wird. Ohne diese Erklärung müsste jemand den Code erst lesen, um die erwarteten Werte zu verstehen.[36]

Dein vollständiger Kommentar für `anlegen(...)` braucht somit:

- vier `@param`-Zeilen: `titel`, `kunde`, `bearbeiter`, `prioritaet`
- eine `@return`-Zeile für das neu angelegte Ticket.

Schreibe ihn jetzt vollständig in deine Datei. Danach nehmen wir eine Methode ohne Parameter, aber mit Rückgabewert — zum Beispiel `getTickets()`. Welche Javadoc-Angabe braucht diese Methode dann auf jeden Fall?

⁂

/**

- Legt ein neues Ticket mit der naechsten freien Id an.
-
- @param titel Titel des Tickets.
- @param kunde Der Kunde des Tickets.
- @param bearbeiter Der zustaendige Bearbeiter.
- @param prioritaet 1 (kritisch) bis 4 (niedrig).
- @return Das neu angelegte Ticket.
  */

Perfekt. Der Kommentar ist vollständig und passend: Er beschreibt die Methode, dokumentiert alle vier Parameter und erklärt den Rückgabewert. Genau das verlangt D1.[37]

Bei `getTickets()` gibt es keine Parameter. Deshalb braucht der Javadoc-Kommentar keinen `@param`-Eintrag, aber unbedingt einen `@return`-Eintrag.

Was gibt deine Methode `getTickets()` genau zurück?

⁂

die tickets

Fast — genauer: Sie gibt die **Liste aller Tickets** zurück, also vermutlich eine `List<Ticket>`, nicht nur ein einzelnes Ticket.

Der entscheidende Javadoc-Teil ist daher:

```
@return Die Liste aller Tickets.
```

Falls du den Typ im Beschreibungstext nennen willst, schreibe ihn in Javadoc als `{@code List<Ticket>}`; die spitzen Klammern dürfen nicht einfach als HTML interpretiert werden.[38]

Wie würdest du jetzt den kurzen Beschreibungssatz direkt über `getTickets()` formulieren?

⁂

<q>

> Falls du den Typ im Beschreibungstext nennen willst, schreibe ihn in Javadoc als {@code List<Ticket>}; die spitzen Klammern dürfen nicht einfach als HTML interpretiert werden.

</q> Wo gehört das dann rein?

Das gehört **in den normalen Beschreibungssatz oder in die `@return`-Zeile** deines Javadoc-Kommentars — nicht in die Methodensignatur.

Zum Beispiel so:

```
/**
 * Gibt alle gespeicherten Tickets als {@code List<Ticket>} zurueck.
 *
 * @return Die Liste aller Tickets.
 */
public List<Ticket> getTickets() {
```

Du musst `{@code List<Ticket>}` aber nicht zwingend verwenden. Diese einfachere Version ist genauso ausreichend:

```
/**
 * Gibt die Liste aller Tickets zurueck.
 *
 * @return Die Liste aller Tickets.
 */
```

Die Schreibweise mit `{@code ...}` brauchst du nur, wenn du den genauen Java-Typ mit den spitzen Klammern im Kommentar nennen möchtest.[39]

Welche der beiden Varianten möchtest du für deine Methode verwenden?

⁂

Empfehlung

Für dieses Projekt: **entferne die drei Setter** und behalte nur Getter, die du tatsächlich brauchst.

- `setTickets(...)`, `setOffeneTickets(...)` und `setKunden(...)` erlauben, dass anderer Code deine kompletten internen Collections ersetzt. Das macht Fehler wahrscheinlicher, weil die Collections zusammenpassen müssen: Ein Ticket in `tickets` sollte z. B. auch korrekt in Queue, Map und Set berücksichtigt sein.
- `getOffeneTickets()` und `getKunden()` solltest du nur behalten, wenn du sie wirklich für Menü, Tests oder eine kommende Aufgabe verwendest. Für Teil 2 sind sie nicht zwingend vorgegeben; die Erweiterungen arbeiten vor allem mit `Auswertung` und der Ticketliste.[40]
- `getTickets()` brauchst du weiterhin, weil `Auswertung` die Liste aller Tickets erhält.[41]

„Vielleicht später“ ist normalerweise kein guter Grund für unbenutzte öffentliche Methoden. Du kannst sie später jederzeit gezielt ergänzen, wenn eine Aufgabe sie wirklich verlangt.

Die gute nächste Entscheidung ist: Wird `getOffeneTickets()` oder `getKunden()` irgendwo in deinem aktuellen `Main`, `Auswertung` oder deinen Testideen benutzt?

⁂

`naechstesErledigen()` dokumentieren

Diese Methode hat keine Parameter und gibt vermutlich nichts zurück (`void`). Ihr Javadoc braucht also nur eine kurze Beschreibung — kein `@param` und kein `@return`. Sie nimmt das nächste offene Ticket aus der Queue, setzt seinen Status auf `ERLEDIGT` und behandelt auch den Fall einer leeren Queue.[42]

Formuliere den Kommentar in eigenen Worten nach diesem Muster:

```
/**
 * ...
 */
public void naechstesErledigen() {
```

Welche zwei Dinge sollten in deinem Satz vorkommen: Was geschieht mit dem nächsten Ticket, und was passiert, wenn kein offenes Ticket vorhanden ist?

⁂

/**

- Nimm das naechste offene Ticket aus der Queue, setzt seinen Status auf ERLEDIGT und behandelt auch den Fall einer leeren Queue.
  */

Inhaltlich sehr gut. Nur die Grammatik im ersten Teil kannst du an den Methodennamen anpassen: Statt „Nimm“ besser „Nimmt“. Außerdem sollte der Kommentar möglichst kurz

bleiben.

```
/**
 * Nimmt das naechste offene Ticket aus der Queue und setzt seinen Status auf ERLEDI
 * Gibt eine Meldung aus, wenn die Queue leer ist.
 */
```

Die Beschreibung deckt beide möglichen Fälle der Methode ab: Ticket erledigen oder leere Queue melden. [44]

Welche Methode dokumentierst du danach sinnvollerweise, wenn sie die letzte Erledigung rückgängig macht?

⁂

letzteErledigungRueckgaengig()

Genau. Auch diese Methode ist `void` und hat keine Parameter. Der Kommentar sollte beschreiben: Sie nimmt das zuletzt erledigte Ticket vom Stack, setzt es auf `OFFEN` und hängt es wieder hinten in die Warteschlange ein. [45]

Formuliere daraus einen kurzen Javadoc-Kommentar in deiner eigenen Sprache:

```
/**
 * ...
 */
public void letzteErledigungRueckgaengig() {
```

Was ist der Unterschied zwischen `poll()` bei der Queue und `pop()` beim Stack?

⁂

/**

- Nimmt das zuletzt erledigte Ticket vom Stack, setzt es auf OFFEN und haengt es wieder hinten in die Warteschlange ein.
  */

Sehr gut. Der Kommentar beschreibt alle drei wichtigen Schritte richtig: vom Stack nehmen, Status zurücksetzen und wieder in die Queue einreihen. [46]

Falls deine Methode auch bei einem leeren Stack eine Meldung ausgibt, ergänze noch einen zweiten Satz dafür — genauso wie bei `naechstesErledigen()`:

```
  * Gibt eine Meldung aus, wenn keine Erledigung rueckgaengig gemacht werden kann.
```

Was bewirkt `pop()` beim Stack im Unterschied zu `poll()` bei der Queue?

⁂

pop() entfernt ein Element aus einem Stack

Fast richtig: `pop()` entfernt **das oberste**, also das **zuletzt hinzugefügte**, Element aus dem Stack und gibt es zurück. Das ist das LIFO-Prinzip: „Last In, First Out“.[47]

Bei eurem Ticketsystem holt `pop()` daher genau das Ticket, das zuletzt als `ERLEDIGT` markiert wurde, damit es rückgängig gemacht werden kann. Was würde passieren, wenn du `pop()` auf einem leeren Stack aufrufst?

⁂

Fehler

Richtig. Bei einem leeren Java-Stack löst `pop()` eine `EmptyStackException` aus. Deshalb solltest du **vorher prüfen**, ob der Stack leer ist, etwa mit `istLeer()` beziehungsweise `isEmpty()`.[62]

Welche Bedingung würdest du am Anfang von `letzteErledigungRueckgaengig()` formulieren, damit dieser Fehler nicht entsteht?

⁂

Der entscheidende Unterschied

Beide Methoden **entfernen ein Element und geben es zurück**. Der Unterschied liegt darin, **welches Element** entfernt wird und was bei einer leeren Collection passiert:

Methode	Typische Collection	Entfernt	Bei leerer Collection
`pop()`	Stack	Das oberste bzw. zuletzt eingefügte Element	Wirft einen Fehler (`EmptyStackException`)
`poll()`	Queue	Das vorderste bzw. zuerst eingefügte Element	Gibt `null` zurück

Für euer Ticketsystem bedeutet das:

- `erledigteTickets.pop()` nimmt das **zuletzt erledigte** Ticket für „Rückgängig“ heraus.
- `offeneTickets.poll()` nimmt das **am längsten wartende offene** Ticket zur Bearbeitung heraus. Die Aufgabenstellung weist ausdrücklich darauf hin, dass bei einer leeren Queue `poll()` den Wert `null` liefert.[67]

Warum ist für eine „Rückgängig“-Funktion ein Stack mit `pop()` passender als eine Queue mit `poll()`?

⁂

Weil Stack nach dem FIFO Prinzip arbeitet

Nicht ganz — hier sind die Prinzipien vertauscht:

- Ein **Stack** arbeitet nach **LIFO**: *Last In, First Out*. Das zuletzt erledigte Ticket wird als erstes rückgängig gemacht.
- Eine **Queue** arbeitet nach **FIFO**: *First In, First Out*. Das Ticket, das am längsten wartet, wird als erstes bearbeitet. [83]

Deshalb passt der Stack zur Rückgängig-Funktion: Meist möchte man die **letzte** Aktion zurücknehmen. Welche Ticketnummer würde bei einem Stack zuerst zurückgenommen, wenn erst #1 und danach #2 erledigt wurden?

⁂

#2

Richtig: **Ticket #2** wird zuerst zurückgenommen, weil es zuletzt auf den Stack gelegt wurde. Anschließend setzt die Methode es auf `OFFEN` und stellt es hinten in die Queue; Ticket #1 bleibt dabei `ERLEDIGT`. [84]

Beschreibe nun in einem Satz, warum die Queue für offene Tickets das FIFO-Prinzip verwendet.

⁂

Weil das erste Ticket, was am längsten wartet zuerst behandelt werden muss

Genau. Präziser formuliert: Eine Queue bearbeitet das Ticket, das **zuerst eingereiht wurde**, auch zuerst — deshalb wartet kein älteres Ticket unnötig hinter neueren Tickets. Das ist FIFO: *First In, First Out*. [85]

Welche Methode verwendest du bei der Queue, um ein neues offenes Ticket **hinten anzufügen**?

⁂

put?

Nicht ganz: `put()` verwendet man bei einer **Map**, um einen Schlüssel mit einem Wert zu speichern, zum Beispiel Bearbeiter und zugehörige Ticketliste. [86]

Für eine Queue wie eure `LinkedList<Ticket>` nimmst du zum Einfügen hinten normalerweise:

```
offeneTickets.add(ticket);
```

Alternativ funktioniert auch:

```
offeneTickets.offer(ticket);
```

Welche Methode nutzt du anschließend, um das erste Ticket aus dieser Queue zu entnehmen?

⁂

poll

Richtig: `poll()` entnimmt das **erste Ticket** aus der Queue und gibt es zurück. Ist die Queue leer, liefert es `null`, statt einen Fehler auszulösen. [102]

Damit hast du den Ablauf von `naechstesErledigen()` verstanden: Welchen Status setzt du dem Ticket direkt nach `poll()`?

⁂

ticket.setStatus(ERLEDIGT);

Richtig gedacht. In deinem Projekt ist der Status offenbar ein `String`, daher braucht `ERLEDIGT` Anführungszeichen:

```
ticket.setStatus("ERLEDIGT");
```

Die Aufgabenstellung nennt `OFFEN`, `IN_BEARBEITUNG` und `ERLEDIGT` als Status-Texte. [118]

Was musst du prüfen, **bevor** du `ticket.setStatus("ERLEDIGT")` aufrufst?

⁂

Tickets pro Bearbeiter

Gern. Diese Methode soll die `Map<String, List<Ticket>>` ausgeben: Der **Schlüssel** ist der Bearbeitername, der **Wert** ist seine Liste von Tickets. Laut Aufgabe sollen damit beispielsweise Anna Berger 3, Lena Fischer 3 und Tom Weber 4 Tickets zugeordnet werden. [119]

Der erste Schritt ist deshalb: Durch welche Collection musst du beim **Hinzufügen eines Tickets** gehen, damit du den Bearbeiter als Schlüssel auslesen kannst?

```
ticket.getBearbeiter()
```

Welche zwei Datentypen stehen in deiner Map links und rechts von `String, ...`?

⁂

Map und List

Fast: `Map` und `List` sind die **Collections**, aber gesucht waren die konkreten **Typen innerhalb der Map**:

```
Map<String, List<Ticket>>
```

- Links: `String` — der Name des Bearbeiters, z. B. `"Anna Berger"`.
- Rechts: `List<Ticket>` — alle Tickets dieser Person. [122] [123]

Wenn du ein Ticket einfügst: Welche Methode brauchst du an der Map, um zu prüfen, ob für `ticket.getBearbeiter()` bereits eine Liste existiert?

⁂

get

`get()` holt den Wert zu einem Schlüssel, prüft aber nicht, ob dieser Schlüssel vorhanden ist. Wenn der Bearbeiter noch fehlt, liefert `get()` nämlich `null`.

Für die Prüfung verwendest du:

```
ticketsProBearbeiter.containsKey(ticket.getBearbeiter())
```

Die Aufgabe empfiehlt genau `containsKey()` und anschließend `put()`, falls es noch keine Ticketliste für den Bearbeiter gibt. [124]

Was musst du mit `put()` als Wert speichern, wenn der Bearbeiter noch nicht in der Map vorkommt?

⁂

können wir bitte weiter machen mit javadoc für die Methode ticketsProBearbeiterAusgeben()

Ja. Die Methode soll die Tickets nach Bearbeiter gruppiert ausgeben. Ein passender, kurzer Javadoc-Kommentar wäre:

```
/**
 * Gibt alle Tickets gruppiert nach Bearbeiter aus.
 */
public void ticketsProBearbeiterAusgeben() {
```

Die Aufgabe verlangt bei Menüpunkt 5 genau diese Ausgabe auf Grundlage einer `Map<String, List<Ticket>>`. [125]

Wenn du zusätzlich ausgibst, wie viele Tickets jede Person hat: Wie würdest du diesen zweiten Aspekt im Kommentar ergänzen?

⁂

mit @parm

Nicht bei dieser Methode: `ticketsProBearbeiterAusgeben()` hat **keine Parameter**, daher brauchst du kein `@param`.

```
/**
 * Gibt alle Tickets gruppiert nach Bearbeiter aus.
 */
public void ticketsProBearbeiterAusgeben() {
```

`@param` verwendest du nur, wenn eine Methode einen Eingabewert besitzt, zum Beispiel:

```
/**
 * Gibt Tickets eines bestimmten Kunden aus.
 *
 * @param kunde Name des Kunden
 */
```

Welche Javadoc-Angabe würdest du verwenden, wenn eine Methode einen Rückgabewert hat?

@return

Richtig: Mit `@return` dokumentierst du, **was eine Methode zurückgibt**.

```
/**
 * Gibt die Anzahl der offenen Tickets zurueck.
 *
 * @return Anzahl der offenen Tickets
 */
public long anzahlOffen() {
```

Bei einer `void`-Methode wie `ticketsProBearbeiterAusgeben()` gibt es kein `@return`, weil sie keinen Wert zurückgibt. Welche Javadoc-Angabe würdest du bei einer Methode verwenden, die einen möglichen Fehler auslösen kann?

D1: Javadoc

D1 verlangt Javadoc für die Klassen `TicketSystem` und `Auswertung` sowie für **alle** `public`-**Methoden**. Jeder Kommentar enthält einen kurzen Beschreibungssatz, pro Parameter ein `@param` und bei Rückgabewerten ein `@return`. Danach erzeugst du die HTML-Dokumentation im Ordner `doku`.[126]

Du hast bereits Kommentare für mehrere `void`-Methoden begonnen. Als nächstes brauchst du einen Klassenkommentar über `TicketSystem`, der seine Aufgabe beschreibt. Schreibe zunächst diesen Anfang selbst:

```
/**
 * Verwaltet ...
 */
public class TicketSystem {
```

Was verwaltet `TicketSystem` insgesamt — nur offene Tickets oder alle Tickets?

⁂

alle

Genau: `TicketSystem` verwaltet **alle Tickets** und nutzt dazu mehrere Collections, etwa Liste, Queue, Stack, Map und Set.[127]

Du kannst den Klassenkommentar daher so schreiben — ohne Umlaute im Quelltext:

```
/**
 * Verwaltet alle Tickets des Ticketsystems mit verschiedenen Collections.
 */
public class TicketSystem {
```

Als nächstes gehört ein Javadoc-Kommentar vor den Konstruktor von `TicketSystem`. Welche Aufgabe hat ein Konstruktor grundsätzlich?

⁂

Das hatten wir schon.

Stimmt, den Konstruktor hatten wir schon. Dann geht es direkt weiter mit der nächsten noch nicht dokumentierten `public`-Methode in `TicketSystem`. Für D1 müssen Klassen und alle öffentlichen Methoden kommentiert sein.[129]

Schick bitte nur die **Methodensignatur** der nächsten Methode, zum Beispiel:

```
public ... methodeName(...) {
```

Dann formulieren wir gezielt den passenden Javadoc-Kommentar dafür.

⁂

public void ticketsSuchen(String suchtext) {

Für diese `void`-Methode brauchst du eine Kurzbeschreibung und genau ein `@param`. Da die Suche laut Zusatzaufgabe Titel unabhängig von Groß- und Kleinschreibung durchsucht, könnte dein Javadoc so aussehen: [130] [131]

```
/**
 * Gibt alle Tickets aus, deren Titel den Suchtext enthaelt.
 *
 * @param suchtext Text, nach dem in den Tickettiteln gesucht wird
 */
public void ticketsSuchen(String suchtext) {
```

Da `void` keinen Wert zurückgibt, gehört hier kein `@return` hinein. Welchen Javadoc-Tag würdest du ergänzen, falls die Methode eine Ausnahme deklarieren würde?

⁂

weiß nicht

Dafür gibt es den Tag `@throws` (gleichbedeutend mit `@exception`). Er beschreibt, **welche Ausnahme** auftreten kann und **unter welcher Bedingung.** [132]

```
/**
 * Liest Daten aus einer Datei.
 *
 * @throws IOException wenn die Datei nicht gelesen werden kann
 */
```

Für deine bisherige Methode `ticketsSuchen()` brauchst du ihn vermutlich nicht, solange sie keine Exception auslöst oder deklariert. Welche drei Javadoc-Tags kennst du jetzt für Parameter, Rückgabewerte und Ausnahmen?

⁂

/**

*

- @param bearbeiter
  */
  public void erstesKritischesTicket(String bearbeiter) {

Der `@param`-Tag ist richtig, aber ihm fehlt noch die Erklärung. Die Methode sucht das erste kritische Ticket eines bestimmten Bearbeiters und gibt entweder dieses Ticket oder eine Meldung aus, falls keines existiert. [146]

```
/**
 * Gibt das erste kritische Ticket eines Bearbeiters aus.
 *
 * @param bearbeiter Name des Bearbeiters
 */
public void erstesKritischesTicket(String bearbeiter) {
```

Da die Methode `void` ist, braucht sie kein `@return`. Welche Information müsste im Kommentar anders sein, wenn die Methode das gefundene Ticket **zurueckgeben** statt ausgeben würde?

⁂

public void kundenAusgeben() {

Diese Methode hat keine Parameter und keinen Rückgabewert. Der Javadoc-Kommentar bleibt deshalb kurz:

```
/**
 * Gibt alle Kunden aus.
 */
public void kundenAusgeben() {
```

Der verwendete `Set<String>` sorgt dabei dafür, dass jeder Kunde nur einmal erscheint.[148]

Was wäre im Kommentar anders, wenn die Methode statt der Ausgabe ein `Set<String>` mit allen Kunden zurückgeben würde?

⁂

Auswertung.java

Gern. `Auswertung` erhält im Konstruktor die Liste aller Tickets und berechnet daraus Auswertungen mit Lambdas und Streams, etwa offene Tickets, kritische Titel, Tickets je Bearbeiter und die durchschnittliche Priorität.[150]

Der Klassenkommentar kann daher so aussehen:

```
/**
 * Wertet Tickets mit Lambdas und Streams aus.
 */
public class Auswertung {
```

Als Nächstes schick die **Methodensignatur** der ersten `public`-Methode aus `Auswertung.java`, dann formulieren wir den passenden Javadoc-Kommentar.

⁂

/**

*

*

- @author Hermann Zimmermann
- @version 1.0
  */

Das sind sinnvolle Angaben für den Klassenkommentar. Es fehlt nur noch eine kurze Beschreibung der Aufgabe von `Auswertung`:

```
/**
 * Wertet Tickets mit Lambdas und Streams aus.
 *
 * @author Hermann Zimmermann
 * @version 1.0
 */
public class Auswertung {
```

`@author` und `@version` sind optional, aber völlig in Ordnung. Für D1 sind besonders die Beschreibung sowie bei Methoden `@param` und `@return` wichtig. [152]

Welche Methode aus `Auswertung` möchtest du als Erstes dokumentieren?

⁂

public Auswertung(List<Ticket> tickets) {

Für den Konstruktor dokumentierst du, dass er die Ticketliste für spätere Auswertungen übernimmt:

```
/**
 * Erstellt eine Auswertung fuer die uebergebene Ticketliste.
 *
 * @param tickets Liste aller auszuwertenden Tickets
 */
public Auswertung(List<Ticket> tickets) {
```

Kein `@return`, weil Konstruktoren keinen Wert zurückgeben.

Welche Methode kommt in deiner Klasse direkt nach dem Konstruktor?

anzahlOffen()

Bei `anzahlOffen()` wird die Zahl der Tickets mit dem Status `OFFEN` zurückgegeben. Deshalb brauchst du einen `@return`-Tag:

```
/**
 * Ermittelt die Anzahl der offenen Tickets.
 *
 * @return Anzahl der Tickets mit dem Status OFFEN
 */
public long anzahlOffen() {
```

Die Aufgabenstellung erwartet dafür einen `long`-Rückgabewert und bei den Beispieldaten den Wert 6. [153]

Warum ist hier `@param` nicht erforderlich?

⁂

Weil es kein Parameter gibt, nur ein return

Genau richtig. `anzahlOffen()` hat keine Eingabe in den Klammern, daher kein `@param`. Sie liefert aber einen Wert vom Typ `long` zurück, daher braucht sie `@return`. [154]

Welche Methode aus `Auswertung` möchtest du als Nächstes mit Javadoc dokumentieren?

⁂

public List<String> kritischeOffeneTitel() {

Diese Methode hat keine Parameter, gibt aber eine Liste von Titeln zurück. Bei generischen Typen wie `List<String>` sollte der Typ im Javadoc mit `{@code ...}` geschrieben werden. [155]

```
/**
 * Ermittelt die Titel aller kritischen offenen Tickets.
 *
 * @return {@code List<String>} mit den Titeln der kritischen offenen Tickets
 */
public List<String> kritischeOffeneTitel() {
```

Welche zwei Bedingungen muss ein Ticket erfüllen, damit sein Titel in dieser Liste landet?

⁂

istOffen und istKritisch

Richtig. Das Ticket muss gleichzeitig `istOffen` sein und `istKritisch` erfüllen — also Status `OFFEN` und Priorität `1` haben. Die Methode filtert deshalb zweimal, bevor sie die Titel sammelt. [156]

Welche Methodensignatur aus `Auswertung` möchtest du als Nächstes dokumentieren?

⁂

public void offeneTicketsAusgeben() {

Da die Methode weder Parameter übernimmt noch etwas zurückgibt, genügt eine Beschreibung:

```
/**
 * Gibt alle offenen Tickets mit Titel und Bearbeiter aus.
 */
public void offeneTicketsAusgeben() {
```

Die Methode filtert die offenen Tickets, formatiert sie mit Titel und Bearbeiter und gibt sie zeilenweise aus. [157]

Welche Methode aus `Auswertung` kommt als Nächstes?

⁂

public List<Ticket> dringendsteOffene(int anzahl) {

Diese Methode hat einen Parameter und gibt eine Ticketliste zurück. Der Kommentar sollte auch erwähnen, dass zuerst nach Priorität und bei Gleichstand nach ID sortiert wird. [159]

```
/**
 * Ermittelt die dringendsten offenen Tickets in sortierter Reihenfolge.
 *
 * @param anzahl Maximale Anzahl der zurueckgegebenen Tickets
 * @return {@code List<Ticket>} mit den dringendsten offenen Tickets
 */
public List<Ticket> dringendsteOffene(int anzahl) {
```

Welche Prioritaetszahl bedeutet in eurem System „kritisch“ und damit besonders dringend?

⁂

Kannst du mir bitte helfen die javadocs für die nächsten Methoden selber zu schreiben?

Ja, gern. Ab jetzt gibst du zuerst selbst einen Entwurf ab, und ich prüfe ihn nur: Beschreibung, passende Tags und Formulierung. Eine Javadoc besteht bei Methoden meist aus:

```
/**
 * Was macht die Methode?
 *
 * @param parametername Wofuer wird dieser Wert gebraucht?
 * @return Was liefert die Methode zurueck?
 */
```

Regel: **jeder Parameter** bekommt `@param`; bei einem Rückgabewert kommt `@return` dazu; bei `void` und Konstruktoren nicht. [161]

Schick die nächste Methodensignatur und deinen Javadoc-Entwurf dazu.

⁂

public List<String> titelVonKunde(String kunde) {

Guter Kandidat zum Selbstschreiben. Lies zuerst nur die Signatur:

- Eingabe: `String kunde` → du brauchst `@param kunde`.
- Ausgabe: `List<String>` → du brauchst `@return`.
- Laut Aufgabe enthält die Liste die Titel der Tickets dieses Kunden und wird alphabetisch sortiert. [176]

Schreibe jetzt den Kommentar in diesem Gerüst aus:

```
/**
 * ...
 *
 * @param kunde ...
 * @return ...
 */
```

Wie würdest du den ersten Beschreibungssatz formulieren?

⁂

Ermittelt die Tickets des Kunden in alphabetischer Reihenfolge.

Fast richtig: Die Methode liefert nicht die ganzen Tickets, sondern nur deren **Titel**. Formuliere daher genauer:

```
Ermittelt die Titel aller Tickets eines Kunden in alphabetischer Reihenfolge.
```

Jetzt ergänze selbst die beiden Zeilen für `@param kunde` und `@return`.

Da habe ich noch Schwierigkeiten mit

Kein Problem — denk bei den Tags immer in zwei Fragen:

- `@param kunde`: **Was gebe ich in die Methode hinein?**
  Hier: den Namen des Kunden, nach dessen Tickets gesucht wird.
- `@return`: **Was kommt aus der Methode heraus?**
  Hier: eine Liste mit den alphabetisch sortierten Tickeltiteln dieses Kunden. Die Aufgabe verlangt genau diese Rückgabeart. [177]

Damit wäre dein vollständiger Kommentar:

```
/**
 * Ermittelt die Titel aller Tickets eines Kunden in alphabetischer Reihenfolge.
 *
 * @param kunde Name des Kunden, dessen Tickets gesucht werden
 * @return Liste mit den alphabetisch sortierten Tickeltiteln des Kunden
 */
```

Du musst den generischen Typ im `@return` nicht zwingend wiederholen, weil er schon in der Methodensignatur steht. Wie würdest du in eigenen Worten erklären, was der Unterschied zwischen einem Parameter und einem Rückgabewert ist?

⁂

<q>

> Tickeltiteln

</q> Ist das ein Rechtschreibfehler?

Nein, das ist kein Rechtschreibfehler. „Tickeltiteln“ ist der Dativ Plural von „Tickeltitel“:

- Nominativ: die Tickeltitel
- Dativ: mit den Tickeltiteln

Der Satz ist also korrekt:

```
@return Liste mit den alphabetisch sortierten Tickeltiteln des Kunden
```

Du könntest auch einfacher schreiben:

```
@return Alphabetisch sortierte Titel der Tickets des Kunden
```

Welche Formulierung findest du für deinen Kommentar verständlicher?

Kommt in den Tag kein Punkt nach dem Satz?

Ja, ein Punkt ist üblich, weil es vollständige Sätze sind. Schreibe also:

```
@param kunde Name des Kunden, dessen Tickets gesucht werden.
@return Liste mit den alphabetisch sortierten Tickeltiteln des Kunden.
```

Auch die erste Beschreibung bekommt einen Punkt:

```
/**
 * Ermittelt die Titel aller Tickets eines Kunden in alphabetischer Reihenfolge.
 *
 * @param kunde Name des Kunden, dessen Tickets gesucht werden.
 * @return Liste mit den alphabetisch sortierten Tickeltiteln des Kunden.
 */
```

In Javadoc ist ein Punkt nicht zwingend technisch nötig, aber für einheitliche und sauber lesbare Dokumentation empfehlenswert. Was würdest du bei einer `void`-Methode anstelle der `@return`-Zeile machen?

⁂

<q>

> Was würdest du bei einer void-Methode anstelle der @return-Zeile machen?

</q> Nichts

Genau richtig. Bei einer `void`-Methode gibt es keine `@return`-Zeile, weil sie keinen Wert zurückgibt.

```
/**
 * Gibt alle offenen Tickets aus.
 */
public void offeneTicketsAusgeben() {
```

Hat die `void`-Methode Parameter, dokumentierst du nur diese mit `@param`. Welchen Tag würdest du bei einer Methode mit zwei Parametern verwenden?

@parm

@parm

Fast: Es heißt `@param` mit **a** nach dem `p`, nicht `@parm`.

Bei zwei Parametern schreibst du zwei Zeilen:

```
@param kunde Name des Kunden.
@param anzahl Maximale Anzahl der Tickets.
```

Jeder Parameter der Methodensignatur bekommt genau einen eigenen `@param`-Tag. Wie würde der erste Tag für einen Parameter namens `status` aussehen?

public boolean hatKritischesOffenesTicket(String kunde) {

Gut, diese Methode hat einen Parameter und liefert einen `boolean` zurück. Daher brauchst du:

```
/**
 * ...
 *
 * @param kunde ...
 * @return ...
 */
```

Sie prüft, ob es für den angegebenen Kunden **mindestens ein Ticket** gibt, das gleichzeitig kritisch und offen ist; dafür wird `anyMatch` verwendet.[192]

Schreibe jetzt zuerst nur den Beschreibungssatz: Was beantwortet diese Methode mit `true` oder `false`?

⁂

Prüft, ob es für den angegebenen Kunden mindestens ein Ticket gibt, das gleichzeitig kritisch und offen ist.

Sehr gut formuliert. Der Beschreibungssatz ist vollständig und präzise. Jetzt fehlen nur noch:

```
@param kunde ...
@return ...
```

Schreibe als Nächstes nur die `@param kunde`-Zeile: Was beschreibt der Wert `kunde`?

Name des Kunden, dessen Tickets gesucht werden.

Genau. Mit dem Tag davor lautet die Zeile:

```
@param kunde Name des Kunden, dessen Tickets gesucht werden.
```

Jetzt fehlt nur noch `@return`: Was bedeutet `true`, und was bedeutet `false` bei dieser Methode?

Sobald die Priorität kirtisch ist un der Status OFFEN lautet, gibt `true`zurück.

Fast richtig. Zwei kleine Korrekturen:

- „kritisch“ statt „kirtisch“
- Es muss zusätzlich zum angegebenen **Kunden** passen.

Als `@return`-Zeile:

```
@return true, wenn der Kunde mindestens ein kritisches offenes Ticket hat, sonst fals
```

Damit ist dein vollständiger Javadoc-Kommentar:

```
/**
 * Prueft, ob es fuer den angegebenen Kunden mindestens ein Ticket gibt,
 * das gleichzeitig kritisch und offen ist.
 *
 * @param kunde Name des Kunden, dessen Tickets gesucht werden.
 * @return true, wenn der Kunde mindestens ein kritisches offenes Ticket hat, sonst
 */
```

Was fehlt in deiner `@return`-Erklärung, wenn du nur Prioritaet und Status nennst, aber den Kunden nicht?

Kunde

Genau. Der Kunde muss in der Erklärung stehen, weil die Methode nicht alle Tickets prüft, sondern nur die Tickets des übergebenen Kunden.

```
@return true, wenn der Kunde mindestens ein kritisches offenes Ticket hat, sonst fals
```

Schick die nächste Methodensignatur aus `Auswertung.java`.

⁂

public Map<String, Long> anzahlProBearbeiter() {

Hier gibt es keine Parameter, aber einen Rückgabewert. Die `Map` ordnet jedem Bearbeiter die Anzahl seiner Tickets zu; die Aufgabe nutzt dafür `groupingBy` und `counting`.[208]

Dein Gerüst lautet also:

```
/**
 * ...
 *
 * @return ...
 */
```

Was liefert die Methode für einen Bearbeiter wie „Anna Berger“ konkret: einen Tickettitel oder eine Anzahl?

⁂

public Map<String, Long> anzahlProBearbeiter() {

return tickets.stream()
.collect(Collectors.groupingBy(ticket -> ticket.getBearbeiter(),
Collectors.counting()));
}

Gut, anhand deines Codes sieht man die Zuordnung klar:

- **Schlüssel** der `Map`: Bearbeitername (`String`)
- **Wert** der `Map`: Anzahl seiner Tickets (`Long`)

Dein Javadoc könnte daher so aussehen:

```
/**
 * Ermittelt die Anzahl der Tickets pro Bearbeiter.
 *
 * @return Map mit Bearbeiternamen als Schluessel und der jeweiligen Ticketanzahl als
 */
```

Dein Code gruppiert alle Tickets nach `getBearbeiter()` und zählt anschließend die Tickets jeder Gruppe.[209]

Was waere bei „Anna Berger“ in deiner Map der Schluessel und was der Wert?

⁂

public double durchschnittPrioritaet() {

return tickets.stream()
.mapToInt(tickets -> tickets.getPrioritaet()) // Holt jede Prioritätszahl.
.average() // Bildet daraus den Durchschnitt.
.orElse(0.0); // Liefert 0.0, falls die Ticketliste leer wäre und daher kein Durchschnitt existiert.
}

Gut erklärt. Die Methode berechnet den Durchschnitt aller Prioritaetswerte; falls keine Tickets vorhanden sind, liefert sie `0.0`.

Versuche den Javadoc selbst mit diesem Geruest:

```
/**
 * ...
 *
 * @return ...
 */
```

Hinweis: Es gibt keine Parameter. Beschreibe im `@return` sowohl den Durchschnitt als auch den Sonderfall einer leeren Ticketliste. Wie würdest du den ersten Satz formulieren?

/**

- Berechnet den Durchschnitt aller Prioritaetswerte; falls keine Tickets vorhanden sind, liefert sie 0.0.
-
- @return Durchschnitt aller Prioritaetswerte, wenn keine Tickets: 0.0
  */

Sehr gut. Inhaltlich stimmt dein Kommentar. Nur den `@return`-Satz würde ich etwas flüssiger formulieren:

```
/**
 * Berechnet den Durchschnitt aller Prioritaetswerte.
 *
 * @return Durchschnitt aller Prioritaetswerte oder 0.0, wenn keine Tickets vorhande
 */
```

Der erste Satz beschreibt die normale Aufgabe der Methode; der `@return`-Tag beschreibt exakt, was zurückkommt — inklusive Sonderfall. Wie würdest du erklären, warum bei einer leeren Liste kein Durchschnitt berechnet werden kann?

public Map<String, Long> offeneTicketsProKunde() {

return tickets.stream()
.filter(istOffen)
.collect(Collectors.groupingBy(ticket -> ticket.getKunde(), Collectors.counting()));
}

Gut, der Code zeigt: Es werden nur offene Tickets berücksichtigt, dann nach Kunde gruppiert und gezählt.

Versuche den Javadoc selbst nach diesem Muster:

```
/**
 * ...
 *
 * @return ...
 */
```

Achte im `@return` darauf, beide Teile der `Map` zu nennen: **Kundenname als Schluessel** und **Anzahl der offenen Tickets als Wert**. Was wäre dein erster Beschreibungssatz?

Es werden nur offene Tickets berücksichtigt, dann nach Kunde gruppiert und gezählt.

Gut beschrieben. Als Javadoc-Satz klingt es noch etwas aktiver und kürzer so:

```
Ermittelt die Anzahl der offenen Tickets pro Kunde.
```

Jetzt fehlt nur noch die `@return`-Zeile. Sie muss erklären: Was ist der **Schluessel** der `Map`, und was ist ihr **Wert**?

String Schlüssel, Long Wert?

Genau. Bei `Map<String, Long>` gilt:

- `String`: der Kundenname als Schluessel
- `Long`: die Anzahl seiner offenen Tickets als Wert

Deine `@return`-Zeile kann daher so lauten:

```
@return Map mit Kundennamen als Schluessel und der Anzahl offener Tickets als Wert.
```

Der vollständige Kommentar wäre:

```
/**
 * Ermittelt die Anzahl der offenen Tickets pro Kunde.
```

```
 *
 * @return Map mit Kundennamen als Schluessel und der Anzahl offener Tickets als Wer
 */
```

Warum ist der Wert hier `Long` und nicht `Ticket`?

Weil Collectors.counting Long zurückgibt

Genau richtig. `Collectors.counting()` zählt die Elemente einer Gruppe und liefert das Ergebnis als `Long` zurück. Deshalb ist der Map-Wert `Long` und nicht `Ticket`.

```
Map<String, Long>
```

- `String`: Kundenname
- `Long`: Anzahl der offenen Tickets

Welche Methode aus `Auswertung.java` möchtest du als Nächstes dokumentieren?

Alle Methoden aus Asuwertung.java sind soweit dokumentiert. Mir ist aufgefallen das offeneTicketsProKunde() nie benutzt wurde

Das ist ein wichtiger Fund: Eine unbenutzte Methode ist kein Fehler, aber ihre Funktion wird im Programm aktuell nicht sichtbar. Damit die Auswertung tatsächlich ausgegeben wird, muss `offeneTicketsProKunde()` in `Main.java` aufgerufen und ihr Rückgabewert ausgegeben werden.

Prüfe als Nächstes in `Main.java`, wo die anderen Methoden von `Auswertung` aufgerufen werden. Dort wäre ein passender Aufruf zum Beispiel:

```
System.out.println(auswertung.offeneTicketsProKunde());
```

Noch lesbarer wäre eine Überschrift davor:

```
System.out.println("Offene Tickets pro Kunde:");
System.out.println(auswertung.offeneTicketsProKunde());
```

Welche bereits verwendete Auswertungsmethode in deiner `Main.java` eignet sich als Vorlage für diesen zusätzlichen Aufruf?

<q>

> Welche bereits verwendete Auswertungsmethode in deiner Main.java eignet sich als Vorlage für diesen zusätzlichen Aufruf?

</q> Wie ist das gemeint?

Gemeint ist: Suche in `Main.java` eine Methode von `Auswertung`, die bereits **aufgerufen und ausgegeben** wird. Deren Aufbau kannst du kopieren und nur den Methodennamen ersetzen. [210]

Zum Beispiel, wenn dort schon etwas in dieser Art steht:

```
System.out.println(auswertung.ticketsProStatus());
```

dann ergänzt du nach demselben Muster:

```
System.out.println(auswertung.offeneTicketsProKunde());
```

Die vorhandene Zeile ist also deine **Vorlage für die Schreibweise** – nicht unbedingt inhaltlich dieselbe Auswertung. Welche `auswertung.`-Aufrufe findest du in deiner `Main.java`?

⁂

Noch eine Anmerkung: In dem Aufgabenblatt steht Richtwert 40min für die Aufgabe Javadoc schreiben und erzeugen, das nehme ich an ist aber ein Richtwert, wenn man weiß wie man das macht. Ich habe jetzt dafür fast einen halben Schultag gebraucht.

Ja, das ist **vollkommen nachvollziehbar**. Ein Richtwert von 40 Minuten beschreibt meist die reine Bearbeitungszeit, wenn Javadoc, Rückgabetypen und das Erzeugen der Dokumentation bereits sicher beherrscht werden. Beim ersten bewussten Durcharbeiten gehört dagegen auch Lesen, Verstehen, Nachschlagen, Formulieren und Kontrollieren dazu.

Gerade bei eurer Aufgabe steckt mehr dahinter als „Kommentare schreiben“: Du musstest den Zweck jeder Methode erfassen, Parameter und Rückgabewerte korrekt beschreiben und bei `Map<String, Long>` verstehen, wofür Schlüssel und Wert stehen. Das ist echte fachliche Arbeit, keine verlorene Zeit. [215]

Wahrscheinlich wird derselbe Aufgabentyp beim nächsten Mal deutlich schneller gehen, weil du jetzt ein wiederverwendbares Muster kennst:

```
/**
 * Beschreibt kurz, was die Methode tut.
```

```
 *
 * @param ... beschreibt den Eingabewert
 * @return beschreibt das Ergebnis
 */
```

Als nächsten kleinen Schritt: Formuliere einmal in eigenen Worten, **welcher Teil der Javadoc-Aufgabe heute am meisten Zeit gekostet hat.**

⁂

Nochmal das ins Gedächtnis rufen und verstehen was die Methode eigentlich macht und das dann formulieren, für jede einzelne Methode

Genau – und das ist vermutlich **der eigentliche Lernteil** der Aufgabe. Du hast nicht nur Kommentare ergänzt, sondern jede Methode gedanklich erneut nachvollzogen:

1. Welche Daten kommen hinein?
2. Was passiert mit ihnen im Methodenrumpf?
3. Welches Ergebnis kommt heraus?
4. Wie lässt sich das kurz und eindeutig formulieren?

Wenn das bei mehreren Methoden noch nicht automatisiert ist, sind 40 Minuten kaum realistisch. Beim nächsten Mal kannst du zuerst pro Methode einen Rohsatz in Alltagssprache notieren, etwa: „Diese Methode filtert offene Tickets und zählt sie pro Kunde.“ Danach wird daraus viel leichter ein sauberer Javadoc-Kommentar.[216]

Welche der dokumentierten Methoden konntest du heute am schnellsten erklären – und woran lag das?

⁂

<q>

> Welche der dokumentierten Methoden konntest du heute am schnellsten erklären – und woran lag das?

</q> weiß ich nicht mehr, alle ungefähr gleich

Das ist auch eine sinnvolle Beobachtung: Wenn alle Methoden etwa gleich viel Zeit brauchen, lag die Herausforderung vermutlich nicht an einer einzelnen komplizierten Methode, sondern am neuen Arbeitsschritt selbst – Code lesen, verstehen und präzise dokumentieren.[217]

Das wird mit Wiederholung schneller. Ein hilfreiches Vorgehen für künftige Aufgaben ist, pro Methode erst drei Stichpunkte zu notieren:

```
Eingabe: Was bekommt die Methode?
Verarbeitung: Was macht sie damit?
Ergebnis: Was gibt sie zurück?
```

Aus diesen drei Punkten lässt sich der Javadoc-Text fast direkt ableiten. Welche dieser drei Fragen würdest du künftig zuerst beantworten, wenn du eine Methode dokumentierst?

⁂

In IntelliJ erzeugen

1. Klicke oben im Menü auf **Tools**.
2. Wähle **Generate Javadoc…**.
3. Wähle bei **Output directory** den Ordner `doku` in deinem Projekt aus bzw. trage ihn ein, zum Beispiel:
   ```
   <Projektordner>/doku
   ```
4. Wähle den Bereich aus, der dokumentiert werden soll – für diese Aufgabe vermutlich das Package mit `Ticket`, `Auswertung`, `Main` und `Beispieldaten`.
5. Lass **Generate index** aktiviert.
6. Klicke auf **Generate**. IntelliJ startet damit das Javadoc-Werkzeug des JDK und legt die HTML-Dateien im ausgewählten Ausgabeordner ab.[218]

Danach sollte der Ordner ungefähr so aussehen:

```
doku/
├── index.html
├── allclasses-index.html
├── index-files/
└── <dein-package>/
```

Öffne anschließend `doku/index.html` im Explorer per Doppelklick im Browser. `index.html` ist die Startseite der erzeugten Javadoc-Dokumentation.[219]

Falls IntelliJ eine Fehlermeldung zeigt: Kopiere bitte den genauen roten Text aus dem **Run**-Fenster hier hinein. Welchen Pfad schlägt IntelliJ bei dir aktuell als Ausgabeordner vor?

⁂

Ursache

Das ist sehr wahrscheinlich ein **Zeichenkodierungsproblem**, nicht ein Fehler in deinen Javadoc-Kommentaren. In deinem Pfad steht der Windows-Benutzerordner:

```
C:\Users\Schüler\...
```

Das ü kann beim Erzeugen bzw. Einlesen der temporären Datei `javadoc_args` falsch kodiert werden. Genau dieser Fehler `Input length = 1` tritt bei Javadoc in IntelliJ unter Windows häufig bei Sonderzeichen im Pfad auf. [237] [238]

Schnellster Versuch

Öffne erneut **Tools → Generate Javadoc…** und trage bei **Other command line arguments** Folgendes ein:

```
-encoding UTF-8 -docencoding UTF-8 -charset UTF-8
```

Dann erneut erzeugen. Diese Optionen legen fest, dass Javadoc die Java-Quelldateien und die erzeugten HTML-Dateien als UTF-8 behandelt. [239]

Wenn es weiter scheitert

Dann liegt es sehr wahrscheinlich am ü im Benutzerpfad. Verschiebe oder kopiere das Projekt testweise in einen Pfad ohne Umlaute, etwa:

```
C:\JavaProjekte\NetStarTicketsystem
```

Setze danach den Ausgabeordner auf:

```
C:\JavaProjekte\NetStarTicketsystem\doku
```

Die Startseite sollte anschließend als `doku\index.html` entstehen. [238] [240]

Führe zuerst den UTF-8-Versuch aus: Welche vollständige Meldung erscheint danach im **Run**-Fenster?

⁂

Der Restfehler

Der Ausgabeordner ist jetzt zwar ohne Umlaute, aber IntelliJ erzeugt weiterhin die Argumentdatei hier:

```
C:\Users\Schüler\AppData\Local\Temp\javadoc_args
```

Darin steckt noch immer das `ü`. Javadoc kann offenbar genau diese temporäre Datei nicht korrekt einlesen; deshalb ändern die UTF-8-Optionen und der neue Ausgabeordner den Fehler nicht. [256]

Dauerhafte Lösung

1. In IntelliJ: **Help → Edit Custom VM Options…**
2. Ganz unten diese Zeile ergänzen:

```
-Dfile.encoding=UTF-8
```

3. Datei speichern.
4. IntelliJ **vollständig schließen und neu starten**.
5. Danach wieder **Tools → Generate Javadoc…** ausführen, mit:
   - Ausgabeordner: `C:\JavaProjekte\NetStarTicketsystem\doku`
   - Zusätzliche Argumente:

     ```
     -encoding UTF-8 -docencoding UTF-8 -charset UTF-8
     ```

JetBrains empfiehlt bei genau diesem Fehler sowohl einen Pfad ohne Sonderzeichen als auch die VM-Option `-Dfile.encoding=UTF-8` und einen IDE-Neustart.[256]

Falls IntelliJ beim Speichern der VM-Options-Datei nach einem Neustart fragt, führe diesen durch und starte erst dann die Javadoc-Erzeugung erneut. Welche Erweiterung hat die Datei, die sich bei **Edit Custom VM Options** geöffnet hat?

⁂

Ich denke, dass es daran liegt, dass der Temp Ordner trotzdem in @C:\Users\Schüler\AppData\Local\Temp\javadoc_args erstellt wird. Wie kann man das beheben?

Ja, genau: Der relevante Pfad ist jetzt nicht mehr dein Projektordner, sondern die von IntelliJ erzeugte Datei `javadoc_args` im Temp-Ordner. Weil darin `C:\Users\Schüler\...` vorkommt, bleibt das ü die wahrscheinliche Ursache. Bei Javadoc ab Java 9 werden solche Argumentdateien als UTF-8 gelesen; enthält der Pfad Zeichen, die beim Erzeugen anders kodiert wurden, kann `Input length = 1` entstehen.[257] [258]

Temp-Ordner umstellen

Am sichersten ist es, IntelliJ selbst einen Temp-Ordner ohne Umlaute zu geben:

1. Lege im Explorer diesen Ordner an:

   ```
   C:\Temp
   ```

2. Öffne in IntelliJ:
   Help → Edit Custom VM Options…

3. Ergänze ganz unten diese **eine** Zeile:

```
-Djava.io.tmpdir=C:\Temp
```

4. Speichern und IntelliJ **vollständig schließen**.
5. IntelliJ neu starten und die Javadoc-Erzeugung erneut ausführen.

`java.io.tmpdir` bestimmt den Ordner für temporäre Dateien einer Java-Anwendung; die Änderung muss beim Start der JVM gesetzt sein, daher ist der Neustart wichtig. [259] [260]

Dann sollte der Befehl am Ende so aussehen:

```
@C:\Temp\javadoc_args
```

Wenn stattdessen weiterhin `@C:\Users\Schüler\AppData\Local\Temp\javadoc_args` erscheint, ist der nächste Weg, die Windows-Umgebungsvariablen **TEMP** und **TMP** für dein Benutzerkonto auf `C:\Temp` zu setzen und dich danach einmal ab- und wieder anzumelden. [261]

Welche Pfadangabe steht im erzeugten Javadoc-Befehl direkt hinter dem `@`, nachdem IntelliJ neu gestartet wurde?

⁂

"C:\Program Files\Java\jdk-25\bin\javadoc.exe" -protected -splitindex -author -version javadoc -d doku -encoding UTF-8TicketSystem.java Auswertung.java -d C:\JavaProjekte\NetStarTicketsystem\doku @C:\Temp\javadoc_args

Fehler: Input length = 1 kann nicht gelesen werden

"javadoc" finished with exit code 1

Deine Vermutung war richtig – der Temp-Pfad ist nun behoben: Die Argumentdatei liegt bei `@C:\Temp\javadoc_args`. Der aktuelle Fehler hat eine **andere Ursache.** [276]

Fehler im Befehl

In deinem Befehl steht dieser Teil:

```
javadoc -d doku -encoding UTF-8TicketSystem.java Auswertung.java
```

Das darf dort nicht stehen. Insbesondere fehlt zwischen `UTF-8` und `TicketSystem.java` ein Leerzeichen, außerdem wurde `javadoc` innerhalb des Aufrufs ein zweites Mal eingefügt. Genau diese Angaben gehören laut Aufgabenblatt als eigenständiger Konsolenbefehl in die Eingabeaufforderung, nicht in das Feld **Other command line arguments** von IntelliJ. [277]

IntelliJ-Einstellungen korrigieren

1. Öffne **Tools → Generate Javadoc…**
2. Leere das Feld **Other command line arguments** vollständig.
3. Setze nur den Ausgabeordner auf:

   ```
   C:\JavaProjekte\NetStarTicketsystem\doku
   ```

4. Wähle die Klassen bzw. das Package mit `TicketSystem` und `Auswertung` aus.
5. Klicke **Generate**.

IntelliJ fügt `javadoc`, `-d`, die Java-Dateien und die temporäre `@...javadoc_args`-Datei selbst hinzu – daher darfst du diese Bestandteile nicht zusätzlich in das Argumentfeld schreiben.[278]

Wenn du die Kodierung explizit setzen möchtest, darf in **Other command line arguments** ausschließlich dies stehen:

```
-encoding UTF-8 -docencoding UTF-8 -charset UTF-8
```

Woran erkennst du im nächsten erzeugten Befehl, dass `javadoc` nicht mehr versehentlich doppelt enthalten ist?

⁂

image.jpg

Das Fenster sieht jetzt korrekt aus:

- **Output directory:** `C:\JavaProjekte\NetStarTicketsystem\doku`
- **Command line arguments:** leer
- **Whole project:** ausgewählt
- **Generate index:** aktiviert

Klicke jetzt einfach auf **Generate**. IntelliJ setzt den nötigen Javadoc-Befehl selbst zusammen; in das Feld **Command line arguments** gehört hier nichts hinein.[280]

Falls der Fehler danach weiterhin erscheint, kopiere bitte den neu erzeugten kompletten Befehl samt Fehlermeldung aus dem **Run**-Fenster. Was steht nach dem Klick auf **Generate** in der Konsole?

⁂

"C:\Program Files\Java\jdk-25\bin\javadoc.exe" -protected -splitindex -author -version -d C:\JavaProjekte\NetStarTicketsystem\doku @C:\Temp\javadoc_args

Fehler: Input length = 1 kann nicht gelesen werden

"javadoc" finished with exit code 1

Jetzt ist klar: **Pfade und IntelliJ-Einstellungen sind korrekt.** Der Fehler entsteht beim Einlesen von `C:\Temp\javadoc_args` selbst. Die Datei enthält wahrscheinlich mindestens ein Zeichen, das nicht UTF-8-konform ist – oft ein Umlaut oder Sonderzeichen aus einem Quelltext-/Package-/Pfadnamen.[281]

Das Aufgabenblatt gibt für genau diese Übung eine wichtige Vorgabe: In **Java-Quelltexten und Javadoc-Kommentaren** sollen keine Umlaute verwendet werden. Prüfe deshalb als Erstes die Javadoc-Kommentare in `TicketSystem.java` und `Auswertung.java`: Suche jeweils nach `ä`, `ö`, `ü` oder `ß` und ersetze sie durch `ae`, `oe`, `ue` bzw. `ss`. Auch Sonderzeichen wie typografische Anführungszeichen solltest du vermeiden.[282]

Eine schnelle Kontrolle in IntelliJ:

1. **Strg + Umschalt + F** drücken.
2. Bei *Text to find* nacheinander `ä`, `ö`, `ü` und `ß` eingeben.
3. Den Suchbereich auf **Whole Project** lassen.
4. Jeden Fund im Java-Code oder Javadoc ersetzen.
5. Anschließend Javadoc erneut erzeugen.

Wenn die Suche keinen Treffer zeigt, prüfe bitte, ob die Paketzeile oder ein Ordner unter `src` einen Umlaut enthält. Welches Zeichen oder welcher Fund wird bei der Suche nach `ü` angezeigt?

⁂

Gilt das auch für Kommentare?

Ja – **auch für Kommentare**, besonders für Javadoc-Kommentare mit `/** ... */`. Sie sind Teil der Java-Quelldatei und werden von `javadoc` eingelesen. Das Aufgabenblatt sagt ausdrücklich: Im Quelltext, **auch in Javadoc-Kommentaren**, keine Umlaute verwenden.[284]

Ersetze also:

```
// Fügt ein Ticket hinzu
```

durch:

```
// Fuegt ein Ticket hinzu
```

und in Javadoc zum Beispiel:

```
/**
 * Ermittelt die Anzahl der offenen Tickets pro Kunde.
 */
```

durch:

```
/**
 * Ermittelt die Anzahl der offenen Tickets pro Kunde.
 */
```

Hier gibt es zwar bereits keine Umlaute, aber bei Wörtern wie „für“, „zurück“, „größte“ oder „überprüft“ musst du `fuer`, `zurueck`, `groesste` und `ueberprueft` schreiben. README-Dateien und Testprotokolle dürfen dagegen Umlaute enthalten.[284]

Wo würdest du bei deinen neu geschriebenen Javadocs am ehesten noch einen Umlaut vermuten?

⁂

"C:\Program Files\Java\jdk-25\bin\javadoc.exe" -protected -splitindex -author -version -d C:\JavaProjekte\NetStarTicketsystem\doku @C:\Temp\javadoc_args

Fehler: Input length = 1 kann nicht gelesen werden

"javadoc" finished with exit code 1

Dann ist die Fehlerquelle **nicht eindeutig durch Umlaute im Java-Code erklärt.** Der Befehl selbst sieht jetzt sauber aus; das Problem steckt vermutlich im Inhalt der automatisch erzeugten Datei `C:\Temp\javadoc_args`, die IntelliJ an `javadoc.exe` übergibt.[285]

Als nächstes brauchen wir den Inhalt dieser Datei, nicht noch einen weiteren Erzeugungsversuch:

1. Öffne den Explorer.
2. Gib oben in die Adressleiste ein:

   ```
   C:\Temp
   ```

3. Suche die Datei `javadoc_args`.

4. Rechtsklick → **Öffnen mit** → Editor.
5. Kopiere den gesamten Inhalt hier hinein.

Wichtig: Die Datei kann nach einem erneuten Versuch wieder überschrieben oder gelöscht werden. Öffne sie daher direkt nach der Fehlermeldung. Daraus lässt sich erkennen, **welche konkrete Datei oder welcher konkrete Pfad** das Zeichenkodierungsproblem auslöst. [286]

Welche Zeilen stehen in `C:\Temp\javadoc_args`?

⁂

es gibt keine javadoc_args datei in C:\Temp

Das ist normal: IntelliJ legt `javadoc_args` nur kurz für den Aufruf an und löscht sie danach wieder. Deshalb kannst du sie nach dem Fehler oft nicht mehr öffnen. [288]

Der praktikable Weg ist jetzt, IntelliJ zu umgehen und die Dokumentation direkt in der Windows-Eingabeaufforderung zu erzeugen. Damit wird keine IntelliJ-Argumentdatei verwendet.

1. Öffne im Explorer deinen Projektordner:

   ```
   C:\JavaProjekte\NetStarTicketsystem
   ```

2. Klicke in die Adressleiste, tippe:

   ```
   cmd
   ```

   und drücke Enter.

3. Gib zuerst ein:

   ```
   dir
   ```

   Prüfe, ob dort `TicketSystem.java` und `Auswertung.java` angezeigt werden.

4. Falls ja, führe diesen Befehl aus:

   ```
   javadoc -d doku -encoding UTF-8 TicketSystem.java Auswertung.java
   ```

Dieser Befehl entspricht dem Vorschlag im Aufgabenblatt und erzeugt die HTML-Dateien im Unterordner `doku`; dort öffnest du anschließend `index.html`. [289]

Wenn `dir` die beiden Dateien **nicht** zeigt, kopiere bitte nur die Liste der angezeigten Ordner und Java-Dateien hier hinein. Welche Ausgabe zeigt `dir` in deinem Projektordner?

⁂

Microsoft Windows [Version 10.0.26300.9711]

(c) Microsoft Corporation. Alle Rechte vorbehalten.

C:\Users\Schüler\Documents\IntelliJ\NetStar Ticketsystem>dir
Datenträger in Laufwerk C: ist Windows-SSD
Volumeseriennummer: C26A-47B0

Verzeichnis von C:\Users\Schüler\Documents\IntelliJ\NetStar Ticketsystem

14.10.2026 13:55 <DIR> .
12.10.2026 08:27 <DIR> ..
12.10.2026 08:27 352 .gitignore
15.10.2026 09:47 <DIR> .idea
14.10.2026 13:55 <DIR> doku
12.10.2026 08:28 433 NetStar Ticketsystem.iml
12.10.2026 08:39 <DIR> out
15.10.2026 09:52 <DIR> src
2 Datei(en), 785 Bytes
6 Verzeichnis(se), 57.056.006.144 Bytes frei

C:\Users\Schüler\Documents\IntelliJ\NetStar Ticketsystem>javadoc -d doku -encoding UTF-8 TicketSystem.java Auswertung.java
Der Befehl "javadoc" ist entweder falsch geschrieben oder
konnte nicht gefunden werden.

C:\Users\Schüler\Documents\IntelliJ\NetStar Ticketsystem>

Die Eingabeaufforderung befindet sich noch im **alten Projektordner** unter `C:\Users\Schüler\...`. Außerdem findet Windows `javadoc` nicht, weil der JDK-Ordner `C:\Program Files\Java\jdk-25\bin` nicht in der PATH-Umgebungsvariable steht. IntelliJ findet die Datei trotzdem, weil es den vollständigen Pfad kennt. [290]

Für den schnellen Test nutze den vollständigen Pfad zu `javadoc.exe`. Deine Java-Dateien liegen sehr wahrscheinlich im Ordner `src`, also führe jetzt diese zwei Befehle nacheinander aus:

```
cd /d C:\JavaProjekte\NetStarTicketsystem\src
```

```
"C:\Program Files\Java\jdk-25\bin\javadoc.exe" -d ..\doku -encoding UTF-8 TicketSyste
```

`-d ..\doku` bedeutet: „Erzeuge die Dokumentation im Ordner `doku`, der eine Ebene oberhalb von `src` liegt.“ Javadoc verarbeitet Java-Quelldateien und erzeugt HTML-Dokumentation im mit `-d` festgelegten Zielordner. [291]

Danach sollte diese Datei existieren:

```
C:\JavaProjekte\NetStarTicketsystem\doku\index.html
```

Führe zuerst nur den `cd`-Befehl aus: Was steht danach links vor dem `>` in der Eingabeaufforderung?

⁂

Microsoft Windows [Version 10.0.26300.9711]

(c) Microsoft Corporation. Alle Rechte vorbehalten.

C:\JavaProjekte\NetStarTicketsystem>dir
Datenträger in Laufwerk C: ist Windows-SSD
Volumeseriennummer: C26A-47B0

Verzeichnis von C:\JavaProjekte\NetStarTicketsystem

14.10.2026 13:59 <DIR> .
14.10.2026 13:59 <DIR> ..
14.10.2026 13:59 <DIR> doku
0 Datei(en), 0 Bytes
3 Verzeichnis(se), 57.060.548.608 Bytes frei

C:\JavaProjekte\NetStarTicketsystem>javadoc -d doku -encoding UTF-8 TicketSystem.java Auswertung.java
Der Befehl "javadoc" ist entweder falsch geschrieben oder
konnte nicht gefunden werden.

C:\JavaProjekte\NetStarTicketsystem>

Du bist jetzt im richtigen **Zielordner**, aber dort befinden sich nur `doku` und keine Java-Dateien. Die Quelldateien sind sehr wahrscheinlich noch im ursprünglichen IntelliJ-Projekt unter:

```
C:\Users\Schüler\Documents\IntelliJ\NetStar Ticketsystem\src
```

Du musst den Befehl außerdem mit dem vollständigen Pfad zu `javadoc.exe` starten, weil Windows `javadoc` nicht über `PATH` findet.[309]

Gib jetzt genau diesen einzelnen Befehl ein:

```
"C:\Program Files\Java\jdk-25\bin\javadoc.exe" -d doku -encoding UTF-8 "C:\Users\Schü
```

Der Befehl verwendet die beiden Quelldateien direkt über ihren vollständigen Pfad und schreibt die HTML-Dokumentation in den aktuellen Unterordner `doku`. Das entspricht dem Javadoc-Aufruf aus dem Aufgabenblatt, nur mit expliziten Pfaden.[310]

Welche Meldung gibt die Konsole nach diesem Befehl aus?

⁂

1. https://labs.it-ninjas.ch/de/docs/02_java/03_java-grundlagen/08_strings/
2. https://www.w3schools.com/python/python_ref_exceptions.asp
3. https://studio.code.org/de/docs/ide/applab/expressions/toLowerCase
4. https://de.wiki-base.com/7780306-javascript-string-tolowercase
5. http://euklid.mi.uni-koeln.de/c/doc/html/selfhtml-7.0/tecl.htm
6. https://wiki.uni-due.de/jack/index.php?title=ToLowerCase
7. https://www.w3schools.com/python/python_user_input.asp
8. https://www.w3schools.com/python/python_range.asp
9. https://www.w3schools.com/python/python_try_except.asp
10. https://wiki.selfhtml.org/wiki/JavaScript/Objekte/String/toLowerCase
11. https://www.cool-lab.net/faq/wie-verwende-ich-die-methode-string-tolowercase/
12. https://developer.mozilla.org/de/docs/Web/JavaScript/Reference/Global_Objects/String/toLowerCase
13. https://code.mu/de/javascript/manual/string/toLowerCase/
14. https://developer.mozilla.org/de/docs/Learn_web_development/Core/Scripting/Useful_string_methods
15. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
16. Ticket.java
17. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
18. Ticket.java
19. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
20. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
21. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
22. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
23. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
24. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
25. Main.java
26. Main.java
27. NetStar-Ticketsystem-Teil-2-Bonusaufgaben-Aufgabenblatt.pdf
28. NetStar-Ticketsystem-Teil-2-Bonusaufgaben-Aufgabenblatt.pdf
29. NetStar-Ticketsystem-Teil-2-Bonusaufgaben-Aufgabenblatt.pdf
30. NetStar-Ticketsystem-Teil-2-Bonusaufgaben-Aufgabenblatt.pdf
31. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
32. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
33. NetStar-Ticketsystem-Teil-2-Bonusaufgaben-Aufgabenblatt.pdf
34. NetStar-Ticketsystem-Teil-2-Bonusaufgaben-Aufgabenblatt.pdf
35. NetStar-Ticketsystem-Teil-2-Bonusaufgaben-Aufgabenblatt.pdf
36. NetStar-Ticketsystem-Teil-2-Bonusaufgaben-Aufgabenblatt.pdf
37. NetStar-Ticketsystem-Teil-2-Bonusaufgaben-Aufgabenblatt.pdf
38. NetStar-Ticketsystem-Teil-2-Bonusaufgaben-Aufgabenblatt.pdf

39. NetStar-Ticketsystem-Teil-2-Bonusaufgaben-Aufgabenblatt.pdf
40. NetStar-Ticketsystem-Teil-2-Bonusaufgaben-Aufgabenblatt.pdf
41. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
42. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
43. NetStar-Ticketsystem-Teil-2-Bonusaufgaben-Aufgabenblatt.pdf
44. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
45. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
46. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
47. https://learn.microsoft.com/de-de/dotnet/api/system.collections.stack.pop?view=net-10.0
48. https://www.w3schools.com/python/python_ref_exceptions.asp
49. https://developer.mozilla.org/de/docs/Web/JavaScript/Reference/Global_Objects/Array/pop
50. https://www.happycoders.eu/de/algorithmen/java-stack/
51. https://codegym.cc/de/groups/posts/de.190.java-stack-101-eintauchen-in-die-stack-klasse
52. https://www.centron.de/tutorials/stack-in-c-programmieren-schritt-fuer-schritt-anleitung
53. https://www.datacamp.com/de/tutorial/python-pop
54. https://www.u-helmich.de/inf/kursQ1/folge14/folge14-3.html
55. https://www.w3schools.com/python/python_range.asp
56. https://www.w3schools.com/python/python_try_except.asp
57. https://www.w3schools.com/PYTHON/python_user_input.asp
58. https://it-berufe-podcast.de/neuer-pruefungskatalog-fuer-die-ap1-der-it-berufe-ab-2025-it-berufe-podcast-190/
59. https://www.reddit.com/r/informatik/comments/xywi8x/wenn_du_ein_stack_hast_und_ein_element_entfernst/
60. https://www.mathematik.uni-ulm.de/numerik/cpp/ws16/doc/cpp-lecture/stack-tutorial/example01/index.html
61. https://ausbildung-in-der-it.de/lexikon/stack-datenstruktur
62. https://www.happycoders.eu/de/algorithmen/java-stack/
63. Ticket.java
64. Main.java
65. Beispieldaten.java
66. NetStar-Ticketsystem-Teil-2-Bonusaufgaben-Aufgabenblatt.pdf
67. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
68. https://www.w3schools.com/python/python_user_input.asp
69. https://jobtensor.com/Tutorial/Python/de/List-Methods-pop
70. https://developer.mozilla.org/de/docs/Web/JavaScript/Reference/Global_Objects/Array/pop
71. https://www.kuketz-blog.de/messenger-unterschied-zwischen-push-und-polling/
72. https://www.musikexpress.de/der-pop-poll-2025-mitmachen-abstimmen-gewinnen-3115787/
73. https://www.popakademie.de/de/dabei-sein/bandpool/
74. https://www.hosteurope.de/faq/e-mail/e-mailkonfiguration/unterschied-zwischen-pop-imap
75. https://www.w3schools.com/python/python_ref_exceptions.asp

76. https://www.w3schools.com/python/python_range.asp
77. https://www.w3schools.com/python/python_try_except.asp
78. https://it-berufe-podcast.de/neuer-pruefungskatalog-fuer-die-ap1-der-it-berufe-ab-2025-it-berufe-podcast-190/
79. https://www.ibm.com/docs/de/i/7.5.0?topic=designs-using-poll-instead-select
80. https://support.microsoft.com/de-de/outlook/what-is-the-difference-between-pop-and-imap
81. https://www.datacamp.com/de/tutorial/python-pop
82. https://www.reddit.com/r/learnpython/comments/vncjjw/difference_between_pop_and_remove/
83. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
84. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
85. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
86. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
87. https://www.w3schools.com/PYTHON/python_user_input.asp
88. https://en.wikipedia.org/wiki/Put_option
89. https://de.wikipedia.org/wiki/Verkaufsoption
90. https://studyflix.de/wirtschaft/kassa-und-termingeschafte-74
91. https://finanzderivate.info/optionen/put-oder-verkaufsoption/
92. https://www.linguee.de/englisch-deutsch/uebersetzung/put.html
93. https://dictionary.cambridge.org/dictionary/english/put
94. https://www.w3schools.com/PYTHON/python_try_except.asp
95. https://www.w3schools.com/python/python_range.asp
96. https://it-berufe-podcast.de/neuer-pruefungskatalog-fuer-die-ap1-der-it-berufe-ab-2025-it-berufe-podcast-190/
97. https://www.w3schools.com/python/python_ref_exceptions.asp
98. https://www.ig.com/at/trading-glossar/put-definition
99. https://www.kagels-trading.de/short-put-verkaufsoption/
100. https://skilling.com/eu/de/blog/trading-terms/selling-a-put/
101. https://www.gevestor.de/finanzwissen/derivate/optionen/put-optionen-749366.html
102. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
103. https://it-berufe-podcast.de/neuer-pruefungskatalog-fuer-die-ap1-der-it-berufe-ab-2025-it-berufe-podcast-190/
104. https://www.politico.com/polling
105. https://www.politico.com/news/2026/10/10/ship-has-sailed-new-poll-shows-even-an-end-to-the-war-or-lower-gas-prices-may-not-save-republicans-01114385
106. https://strawpoll.com/create/
107. https://www.merriam-webster.com/dictionary/poll
108. https://www.britannica.com/dictionary/poll
109. https://www.dictionary.com/browse/poll
110. https://www.w3schools.com/PYTHON/python_user_input.asp
111. https://www.w3schools.com/python/python_range.asp

112. https://www.w3schools.com/python/python_try_except.asp
113. https://www.w3schools.com/python/python_ref_exceptions.asp
114. https://www.microsoft.com/en-us/microsoft-365/online-surveys-polls-quizzes
115. https://www.poll-maker.com/
116. https://de.wikipedia.org/wiki/Poll_(Köln)
117. https://www.poll-schornsteine.de/
118. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
119. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
120. Main.java
121. Ticket.java
122. Ticket.java
123. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
124. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
125. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
126. NetStar-Ticketsystem-Teil-2-Bonusaufgaben-Aufgabenblatt.pdf
127. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
128. NetStar-Ticketsystem-Teil-2-Bonusaufgaben-Aufgabenblatt.pdf
129. NetStar-Ticketsystem-Teil-2-Bonusaufgaben-Aufgabenblatt.pdf
130. NetStar-Ticketsystem-Teil-2-Bonusaufgaben-Aufgabenblatt.pdf
131. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
132. https://www.oracle.com/technical-resources/articles/java/javadoc-tool.html
133. https://www3.cs.stonybrook.edu/~cse214/Javadoc.htm
134. https://idratherbewriting.com/java-javadoc-tags/
135. https://www.baeldung.com/javadoc
136. https://issues.apache.org/jira/browse/THRIFT-5487
137. https://checkstyle.sourceforge.io/version/10.21.1/checks/javadoc/javadocmethod.html
138. https://www.w3schools.com/PYTHON/python_try_except.asp
139. https://www.w3schools.com/python/python_ref_exceptions.asp
140. https://www.w3schools.com/python/python_range.asp
141. https://www.w3schools.com/PYTHON/python_user_input.asp
142. http://www.javapractices.com/topic/TopicAction.do?Id=171
143. https://stackoverflow.com/questions/5510170/in-javadoc-what-is-the-difference-between-the-tags-throws-and-exception
144. https://courses.cs.cornell.edu/cs212/2008sp/Compiler/Java/javadoc.html
145. https://www.jetbrains.com/help/idea/javadocs.html
146. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
147. NetStar-Ticketsystem-Teil-2-Bonusaufgaben-Aufgabenblatt.pdf
148. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf

149. NetStar-Ticketsystem-Teil-2-Bonusaufgaben-Aufgabenblatt.pdf
150. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
151. NetStar-Ticketsystem-Teil-2-Bonusaufgaben-Aufgabenblatt.pdf
152. NetStar-Ticketsystem-Teil-2-Bonusaufgaben-Aufgabenblatt.pdf
153. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
154. NetStar-Ticketsystem-Teil-2-Bonusaufgaben-Aufgabenblatt.pdf
155. NetStar-Ticketsystem-Teil-2-Bonusaufgaben-Aufgabenblatt.pdf
156. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
157. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
158. NetStar-Ticketsystem-Teil-2-Bonusaufgaben-Aufgabenblatt.pdf
159. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
160. NetStar-Ticketsystem-Teil-2-Bonusaufgaben-Aufgabenblatt.pdf
161. https://www.oracle.com/technical-resources/articles/java/javadoc-tool.html
162. https://www.w3schools.com/python/python_range.asp
163. https://learntutorials.net/de/java/topic/140/java-code-dokumentieren
164. https://sdq.kastel.kit.edu/programmieren/JavaDoc
165. https://www.tutkit.com/de/text-tutorials/13255-javadoc-effizient-nutzen-anleitung-fuer-einsteiger
166. https://openbook.rheinwerk-verlag.de/javainsel/24_004.html
167. https://dbs.cs.uni-duesseldorf.de/lehre/docs/java/javabuch/html/k100313.html
168. https://www.w3schools.com/python/python_ref_exceptions.asp
169. https://www.w3schools.com/python/python_try_except.asp
170. https://www.w3schools.com/PYTHON/python_user_input.asp
171. https://it-berufe-podcast.de/neuer-pruefungskatalog-fuer-die-ap1-der-it-berufe-ab-2025-it-berufe-podcast-190/
172. https://coddy.tech/docs/de/java/methods
173. https://de.wikibooks.org/wiki/Java_Standard:_Javadoc
174. https://dh-cologne.github.io/java-wegweiser/articles/Javadoc.html
175. https://www.jetbrains.com/help/idea/javadocs.html
176. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
177. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
178. https://www.w3schools.com/python/python_ref_exceptions.asp
179. https://www.sofatutor.com/mathematik/videos/wie-schreibe-ich-ein-datum
180. https://www.gutefrage.net/frage/wie-haelt-man-es-mit-der-punktsetzung-nach-einem-datum
181. https://www.reddit.com/r/German/comments/hzxw84/punkt_nach_datum_und_etc/
182. https://korrektur.de/zeichen-punkt
183. https://www.bbc.co.uk/learningenglish/course/lower-intermediate/unit-1/tab/grammar
184. https://www.w3schools.com/python/python_try_except.asp
185. https://www.w3schools.com/python/python_range.asp

186. https://www.w3schools.com/PYTHON/python_user_input.asp
187. https://www.annika-lamer.de/so-schreiben-sie-datum-und-uhrzeit-richtig/
188. https://www.textskizzen.de/datumsangaben-korrekt-schreiben/
189. https://www.deutscheslektorat.de/forum/viewtopic.php?t=22223
190. https://sandra-meinzenbach.de/datum-richtig-schreiben/
191. https://www.duden.de/sprachwissen/rechtschreibregeln/datum
192. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
193. https://www.w3schools.com/PYTHON/python_user_input.asp
194. https://wirtschaftslexikon.gabler.de/definition/kunde-37108
195. https://www.olev.de/k/kunde.htm
196. https://targenio.de/was-bedeutet-kunde-eigentlich/
197. https://www.buchstaben.com/raetsel/kunde
198. https://dictionary.cambridge.org/dictionary/german-english/kunde
199. https://en.pons.com/translate/german-english/Kunde
200. https://www.w3schools.com/python/python_range.asp
201. https://it-berufe-podcast.de/neuer-pruefungskatalog-fuer-die-ap1-der-it-berufe-ab-2025-it-berufe-podcast-190/
202. https://www.w3schools.com/python/python_try_except.asp
203. https://www.w3schools.com/python/python_ref_exceptions.asp
204. https://de.wikipedia.org/wiki/Kunde
205. https://www.dwds.de/wb/Kunde
206. https://www.marketinginstitut.biz/blog/was-ist-ein-kunde/
207. https://www.juraforum.de/lexikon/kunde
208. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
209. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
210. Ticket.java
211. Main.java
212. Beispieldaten.java
213. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
214. NetStar-Ticketsystem-Teil-2-Bonusaufgaben-Aufgabenblatt.pdf
215. Ticket.java
216. Ticket.java
217. Ticket.java
218. https://www.jetbrains.com/help/idea/javadocs.html
219. https://www.oracle.com/java/technologies/javase/javadoc-faq.html
220. https://www.jetbrains.com/help/idea/editing-html-files.html
221. https://stackoverflow.com/questions/1843072/intellij-generate-javadoc-for-methods-and-classes
222. http://www.iditect.com/faq/java/how-to-generate-javadoc-from-command-line.html

223. https://www.javathinking.com/blog/markdown-replacement-for-javadoc-also-for-ides/
224. https://www.javaspring.net/blog/intellij-generate-javadoc-for-methods-and-classes/
225. https://www.javaspring.net/blog/javadoc-auto-generation-for-intellij/
226. https://stackoverflow.com/questions/4592396/how-to-generate-javadoc-from-command-line
227. https://www.javaspring.net/blog/how-to-generate-javadoc-from-command-line/
228. https://stackoverflow.com/questions/24867716/where-do-i-place-generated-javadoc-html-files
229. https://docs.oracle.com/en/java/javase/24/docs/specs/man/javadoc.html
230. https://www.w3schools.com/python/python_ref_exceptions.asp
231. https://www.w3schools.com/python/python_try_except.asp
232. https://www.w3schools.com/python/python_range.asp
233. https://www.w3schools.com/python/python_user_input.asp
234. https://docs.oracle.com/javase/8/docs/technotes/tools/windows/javadoc.html
235. https://docs.oracle.com/en/java/javase/17/docs/specs/man/javadoc.html
236. https://stackoverflow.com/questions/54934173/export-javadoc-to-html-in-intelij
237. https://stackoverflow.com/questions/53248589/intellij-javadoc-error-cannot-read-input-length-1
238. https://intellij-support.jetbrains.com/hc/en-us/community/posts/10611859833490-javadoc-error-cannot-read-Input-length-1
239. https://www.jetbrains.com/help/idea/javadocs.html
240. https://intellij-support.jetbrains.com/hc/en-us/community/posts/10508042264594-Error-when-i-create-javaDoc-cannot-read-Input-length-1
241. https://stackoverflow.com/questions/53449049/maven-javadoc-plugin-error-javadoc-error-cannot-read-input-length-1-with-no
242. https://stackoverflow.com/questions/31177326/how-to-set-encoding-in-intellij-javadoc-generating
243. https://stackoverflow.com/questions/1843072/intellij-generate-javadoc-for-methods-and-classes
244. https://velog.io/@refoli_20/Javadoc-에러-cannot-read-Input-length-1
245. https://www.jetbrains.com/help/idea/javadocs.html.md/
246. https://codingtechroom.com/question/fix-nio-charset-malformedinputexception-input-length-1-error
247. https://www.javaspring.net/blog/intellij-generate-javadoc-for-methods-and-classes/
248. https://www.javaspring.net/blog/javadoc-auto-generation-for-intellij/
249. https://www.javaspring.net/blog/all-inclusive-charset-to-avoid-java-nio-charset-malformedinputexception-input-length-1/
250. https://codingtechroom.com/question/fix-maven-javadoc-plugin-error-non-ascii-characters
251. https://www.w3schools.com/python/python_ref_exceptions.asp
252. https://www.w3schools.com/python/python_try_except.asp
253. https://www.w3schools.com/PYTHON/python_user_input.asp
254. https://www.w3schools.com/python/python_range.asp
255. https://www.reddit.com/r/IntelliJIDEA/comments/doos63/javadoc_issues_hello_guys_recently_i_find_that/

256. https://intellij-support.jetbrains.com/hc/en-us/community/posts/10508042264594-Error-when-i-create-javaDoc-cannot-read-Input-length-1
257. https://stackoverflow.com/questions/53248589/intellij-javadoc-error-cannot-read-input-length-1
258. https://issues.apache.org/jira/browse/MJAVADOC-544
259. https://intellij-support.jetbrains.com/hc/en-us/articles/206544869-Configuring-JVM-options-and-platform-properties
260. https://stackoverflow.com/questions/1924136/environment-variable-to-control-java-io-tmpdir
261. https://knowledge.civilgeo.com/changing-the-windows-temp-and-tmp-directory
262. https://www.jetbrains.com/help/idea/directories-used-by-the-ide-to-store-settings-caches-plugins-and-logs.html
263. https://stackoverflow.com/questions/53449049/maven-javadoc-plugin-error-javadoc-error-cannot-read-input-length-1-with-no
264. https://www.jetbrains.com/help/idea/tuning-the-ide.html
265. https://www.jetbrains.com/help/idea/program-arguments-and-environment-variables.html
266. https://intellij-support.jetbrains.com/hc/en-us/community/posts/115000649644-Javadoc-argfile-encoding-error
267. https://intellij-support.jetbrains.com/hc/en-us/articles/206544519-Directories-used-by-the-IDE-to-store-settings-caches-plugins-and-logs
268. https://www.baeldung.com/intellij-idea-set-jvm-arguments
269. https://gist.github.com/HELWATANY/c86ac31128468267c16ff0ef9f740637
270. https://intellij-support.jetbrains.com/hc/en-us/community/posts/10508042264594-Error-when-i-create-javaDoc-cannot-read-Input-length-1
271. https://stackoverflow.com/questions/36431624/how-to-deal-with-huge-amount-of-temporary-files-generated-by-intellij-idea
272. https://www.w3schools.com/PYTHON/python_user_input.asp
273. https://www.w3schools.com/python/python_ref_exceptions.asp
274. https://www.w3schools.com/python/python_try_except.asp
275. https://www.w3schools.com/python/python_range.asp
276. Ticket.java
277. NetStar-Ticketsystem-Teil-2-Bonusaufgaben-Aufgabenblatt.pdf
278. https://www.jetbrains.com/help/idea/javadocs.html
279. NetStar-Ticketsystem-Tagesprojekt-Aufgabenblatt.pdf
280. image.jpg
281. Ticket.java
282. NetStar-Ticketsystem-Teil-2-Bonusaufgaben-Aufgabenblatt.pdf
283. image.jpg
284. NetStar-Ticketsystem-Teil-2-Bonusaufgaben-Aufgabenblatt.pdf
285. Ticket.java
286. https://intellij-support.jetbrains.com/hc/en-us/community/posts/115000649644-Javadoc-argfile-encoding-error
287. Main.java
288. Ticket.java
289. NetStar-Ticketsystem-Teil-2-Bonusaufgaben-Aufgabenblatt.pdf

290. https://stackoverflow.com/questions/42918400/how-to-run-javadoc-from-windows-10
291. https://docs.oracle.com/en/java/javase/24/docs/specs/man/javadoc.html
292. https://docs.oracle.com/en/java/javase/21/docs/specs/man/javadoc.html
293. https://www.windows-commandline.com/javac-not-recognized-internal-external-command/
294. https://www.theserverside.com/blog/Coffee-Talk-Java-News-Stories-and-Opinions/Java-Not-Recognized-Error-Fix/
295. https://javarevisited.blogspot.com/2012/12/javac-is-not-recognized-as-internal-or-external-command.html
296. https://docs.oracle.com/cd/E19182-01/821-0917/inst_jdk_javahome_t/index.html
297. https://stackoverflow.com/questions/6597562/obtain-and-download-javadoc-jdk-api-documentation-to-a-local-file-for-offline
298. https://www.reddit.com/r/javahelp/comments/1mqfenr/cmd_does_not_recognize_the_java_version_command/
299. https://www.javaspring.net/blog/how-to-generate-javadoc-from-command-line/
300. https://www.javathinking.com/blog/javac-is-not-recognized-as-an-internal-or-external-command-operable-program-or-batch-file/
301. https://github.com/redhat-developer/vscode-java/issues/1062
302. https://www.w3schools.com/python/python_ref_exceptions.asp
303. https://www.w3schools.com/python/python_try_except.asp
304. https://www.w3schools.com/python/python_range.asp
305. https://www.w3schools.com/PYTHON/python_user_input.asp
306. https://docs.oracle.com/cd/E19182-01/820-7851/inst_cli_jdk_javahome_t/
307. https://intellij-support.jetbrains.com/hc/en-us/community/posts/360010574119-Maven-not-finding-JavaDoc-command
308. https://www.oracle.com/java/technologies/javase/frequently-asked-questions-doc.html
309. Ticket.java
310. NetStar-Ticketsystem-Teil-2-Bonusaufgaben-Aufgabenblatt.pdf